PRAKTIKUM 1-5

LAPORAN PRAKTIKUM



OLEH:

LEXI MULIA YUNASPI

2511531006

KELAS B

MATA KULIAH PEMOGRAMAN BERORIENTASI OBJEK

DOSEN PENGAMPU : NURFIAH, S.ST, M.KOM

ASISTEN PRAKTIKUM : EZZA ADDINI

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis ucapkan kehadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan Praktikum 1 Pemrograman Berorientasi Objek ini dapat diselesaikan. Laporan ini dibuat sebagai dokumentasi proses praktikum mengenai class, object, constructor, method, serta interaksi objek melalui program Mini Bank berbasis Command Line Interface (CLI). Pada praktikum ini penulis membuat program menggunakan bahasa Java dan Eclipse. Program dasar terdiri dari class Rekening dan Main, kemudian dikembangkan pada bagian tugas dengan penambahan fitur tarik tunai serta penyimpanan beberapa rekening menggunakan ArrayList. Pada laporan ini source code tidak ditampilkan sekaligus dalam satu blok panjang, tetapi dipisahkan menjadi beberapa bagian kecil agar fungsi setiap bagian kode lebih mudah dipahami. Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan agar laporan berikutnya dapat dibuat dengan lebih baik.Semoga laporan ini dapat membantu menjelaskan proses praktikum dan menjadi dokumentasi hasil pengerjaan program.

Padang, 2026

Penulis

PEKAN 1

CLASS, OBJECT, CONSTRUCTOR DAN METHOD

INTERAKSI OBJEK & MANIPULASI STATE (MINI BANK)

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pemrograman Berorientasi Objek merupakan pendekatan pemrograman yang menyusun program berdasarkan objek dan class. Pada Praktikum 1, konsep tersebut diterapkan melalui program Mini Bank sederhana yang dijalankan melalui Command Line Interface (CLI).

Program ini dipilih karena proses perubahan data saldo dapat terlihat secara langsung ketika sebuah method dipanggil.

Sesuai modul praktikum, materi utama yang dipelajari adalah class, object, constructor, dan method. Class digunakan sebagai cetak biru untuk membuat objek, sedangkan object merupakan bentuk nyata dari class yang mempunyai state sendiri. Constructor digunakan ketika objek pertama kali dibuat untuk memberikan nilai awal, sedangkan method digunakan untuk menjalankan perilaku atau operasi terhadap objek.

Pada tahap awal, program hanya mengelola satu rekening aktif. Setelah bagian tugas dikerjakan, program dikembangkan agar mempunyai fitur tarik tunai dan dapat menyimpan lebih dari satu rekening menggunakan ArrayList. Dengan demikian, praktikum tidak hanya memperkenalkan pembuatan class, tetapi juga memperlihatkan bagaimana state sebuah object dapat dimanipulasi melalui interaksi pengguna.

1.2 Tujuan Praktikum

1. Memahami cara membuat class sebagai model dari entitas dunia nyata.
2. Memahami pembuatan object dan state yang dimiliki oleh object.
3. Memahami penggunaan constructor untuk memberikan state awal object.

4. Memahami penggunaan method untuk memanipulasi state melalui program CLI.
5. Mengimplementasikan fitur tarik tunai sesuai aturan pada modul.

6. Mengembangkan program agar dapat menyimpan beberapa rekening menggunakan ArrayList.

1.3 Manfaat Praktikum

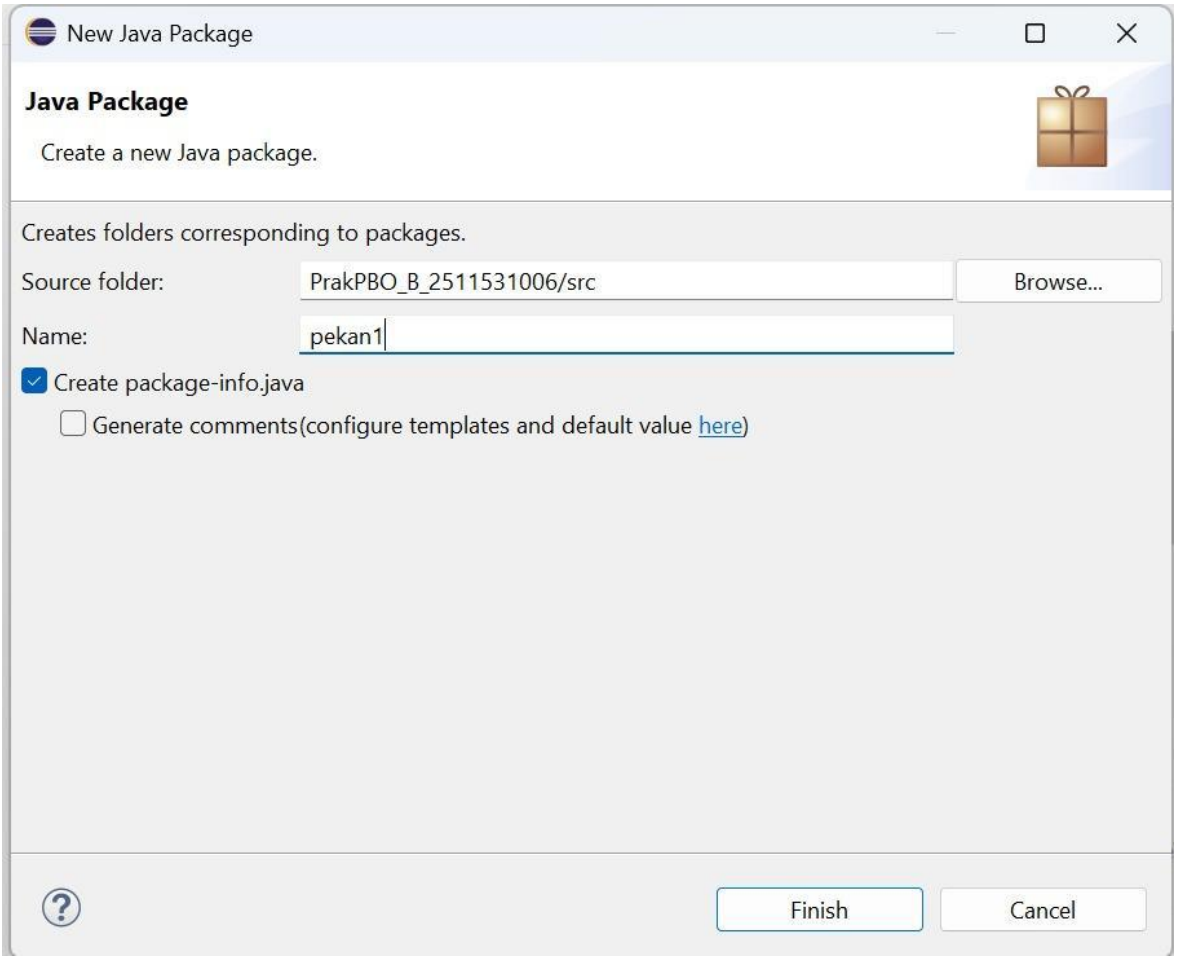
Praktikum ini membantu penulis memahami hubungan antara class, object, constructor, dan method melalui contoh yang sederhana. Selain itu, penulis dapat melihat secara langsung perubahan nilai saldo setelah method setor tunai atau tarik tunai dijalankan. Penggunaan ArrayList pada bagian tugas juga memberikan pemahaman awal mengenai pengelolaan beberapa object dalam satu program.

BAB II

PEMBAHASAN

2.1 Persiapan Project dan Package

Project yang digunakan pada praktikum ini adalah PrakPBO_B_2511531006. Di dalam project terdapat source folder dan beberapa package yang digunakan untuk memisahkan program dasar dengan program tugas. Pada pengerjaan ini terdapat empat class utama yang dibahas,yaitupekan1.Rekening,pekan1.Main,TugasPekan1.Rekening,TugasPekan1.Main.

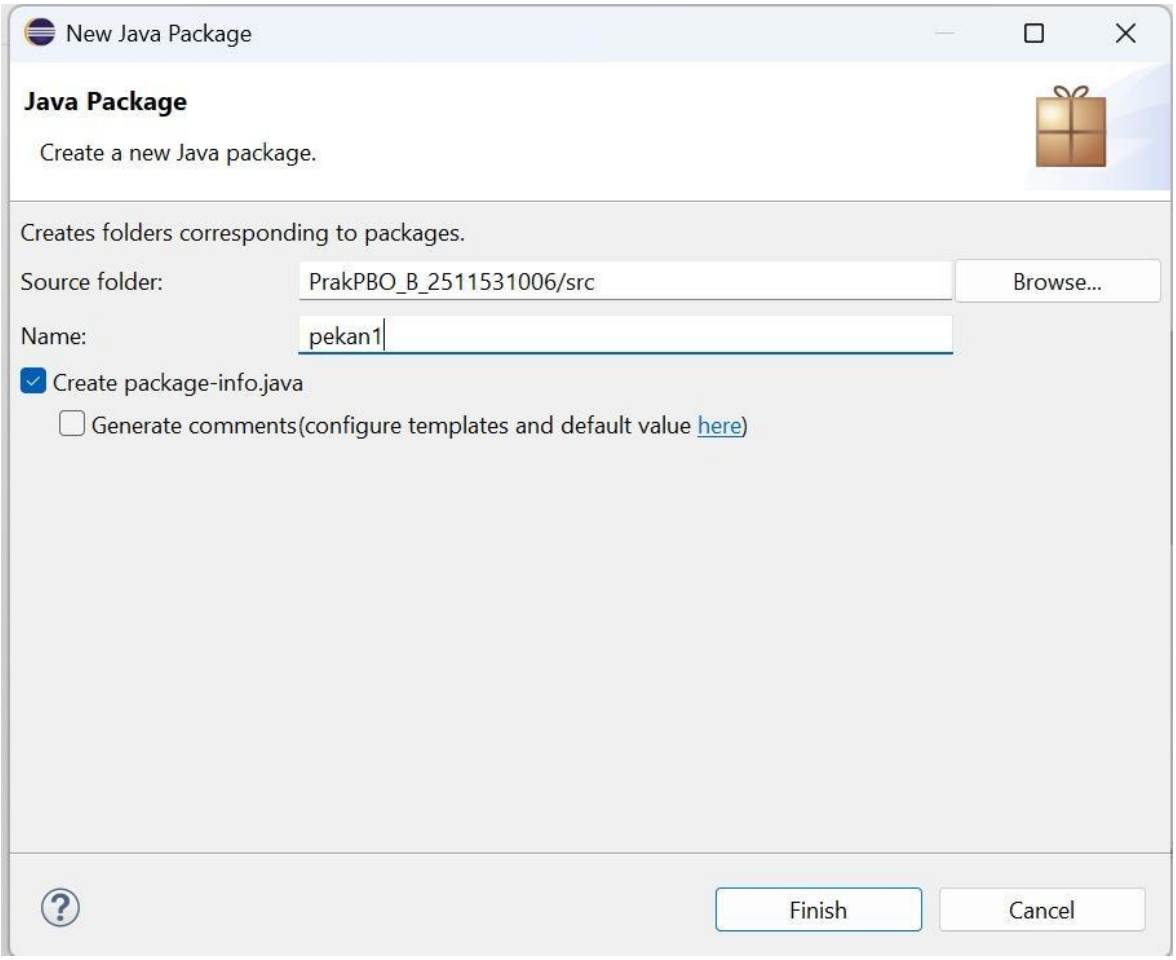


Gambar 2.1 Pembuatan package pekan1

Pada membuat package, bagian Source folder diarahkan ke PrakPBO_B_2511531006/src.

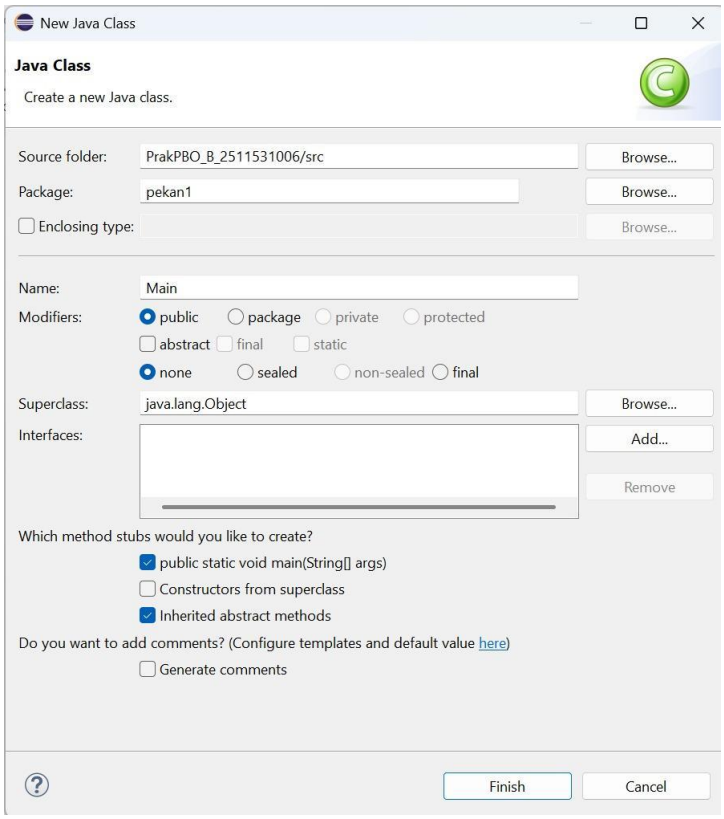
Nama package yang digunakan pada program dasar adalah pekan1. Pilihan Create package-

info.java dapat dibiarkan aktif seperti pada tampilan Eclipse. Setelah tombol Finish dipilih, package akan muncul di dalam folder src dan siap digunakan untuk membuat class.



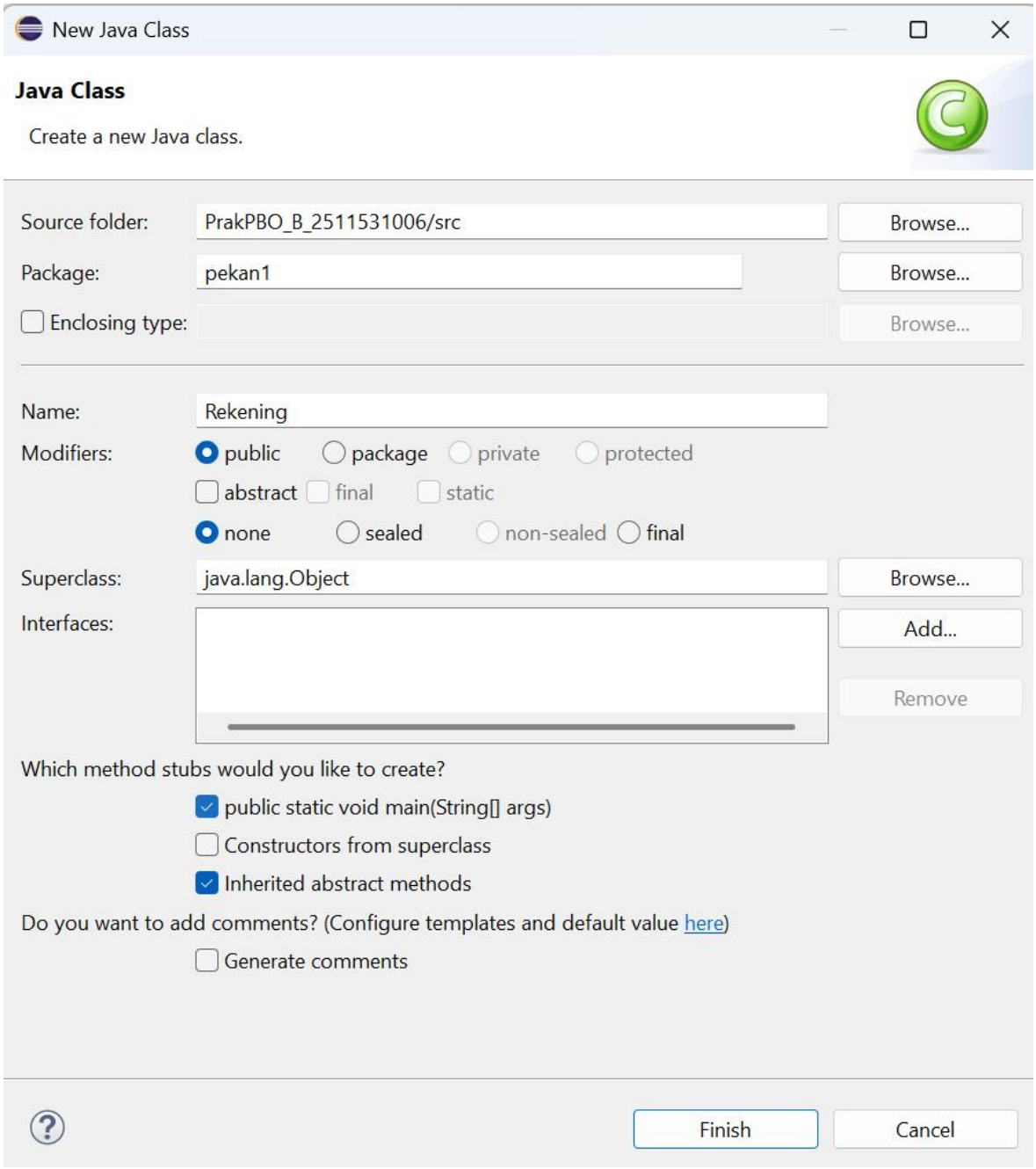
Gambar 2.2 Pembuatan package untuk bagian tugas

Untuk bagian tugas dibuat package TugasPekan1.Package ini digunakan sebagai tempat class hasil pengembangan dari program dasar. Dengan adanya package terpisah, perubahan fitur seperti tarik tunai dan multi-akun tidak bercampur dengan program dasar.



Gambar 2.3 Pembuatan class Rekening

Class Rekening dibuat sebagai class model yang menyimpan data rekening dan menyediakan method untuk melakukan operasi pada saldo. Pada pembuatan class, modifier public dipilih agar class dapat digunakan dari class lain. Method main tidak diperlukan pada class Rekening karena class ini bukan pusat eksekusi program.



Gambar 2.4 Pembuatan class Main

Class Main digunakan sebagai tempat utama program dijalankan. Karena Main memiliki method main, class ini yang menerima input pengguna, menampilkan menu, dan memanggil method pada object Rekening. Pola pembagian ini sesuai dengan

konsep sederhana PBO pada modul, yaitu Rekening sebagai model dan Main sebagai pengatur interaksi pengguna.

2.2 Program Dasar - Class pekan1.Rekening

Class pertama yang dibuat adalah Rekening.java pada package pekan1. Sesuai modul, class ini berfungsi sebagai model utama rekening. Di dalamnya terdapat atribut sebagai state,

constructor untuk memberikan nilai awal, dan beberapa method sebagai behavior yang dapat mengubah atau menampilkan state rekening.

a. Package dan deklarasi class

```
1  package pekan1;
2
3  public class Rekening {
4
```

Baris package pekan1 menunjukkan bahwa class Rekening ditempatkan di dalam package pekan1. Setelah itu dibuat class dengan nama Rekening dan modifier public.

Class ini menjadi cetak biru untuk objek rekening yang nantinya dibuat oleh class Main. Jadi, data dan operasi yang berkaitan dengan rekening diletakkan terlebih dahulu di class ini.

b. Atribut atau state rekening

```
5      String nomorRekening;
6      String namaPemilik;
7      double saldo;
8
```

Tiga baris tersebut merupakan atribut yang menyimpan state dari setiap object Rekening. nomorRekening digunakan untuk menyimpan identitas rekening, namaPemilik menyimpan nama pemilik, sedangkan saldo menyimpan jumlah uang yang dimiliki. Setiap object Rekening dapat mempunyai nilai atribut yang berbeda sehingga satu rekening tidak harus memiliki data yang sama dengan rekening lainnya.

c. Constructor Rekening

```
9      public Rekening(String nomor, String nama, double saldoAwal) {
10         nomorRekening = nomor;
11         namaPemilik = nama;
12         saldo = saldoAwal;
```

Bagian ini merupakan constructor yang akan dijalankan ketika object Rekening

dibuat menggunakan keyword new. Constructor menerima tiga parameter, yaitu nomor rekening, nama pemilik, dan saldo awal. Nilai dari parameter tersebut kemudian dimasukkan ke atribut object. Dengan cara ini, object langsung mempunyai data awal ketika berhasil dibuat.

d. Pesan saat rekening dibuat

```
14          System.out.println("Rekening atas nama " + namaPemilik
15                  + " berhasil dibuat dengan saldo Rp" + saldo);
16      }
```

Kode tersebut digunakan untuk memberikan informasi kepada pengguna bahwa rekening berhasil dibuat. Nama pemilik dan saldo ditampilkan menggunakan nilai atribut yang sudah diisi pada constructor. Output ini juga membantu saat pengujian karena kita dapat melihat bahwa object Rekening benar-benar berhasil dibuat dan mempunyai saldo awal yang sesuai.

e. Method setorTunai

```
18      public void setorTunai(double nominal) {
19          if (nominal > 0) {
20              saldo += nominal;
```

Method setorTunai digunakan untuk menambahkan uang ke saldo rekening.

Parameter nominal menunjukkan jumlah uang yang akan disetorkan. Kondisi if memastikan bahwa nominal harus lebih besar dari nol. Jika syarat terpenuhi, nilai saldo ditambah dengan nominal sehingga state object berubah setelah method dijalankan.

f. Output setor tunai

```
22              System.out.println("Setor tunai Rp" + nominal
23                      + " berhasil. Saldo saat ini: Rp" + saldo);
24          } else {
25              System.out.println("Gagal: Nominal setor harus lebih dari 0!");
26          }
27      }
```

Setelah saldo berhasil ditambah, program menampilkan pesan bahwa transaksi setor berhasil beserta saldo terbaru. Jika nominal yang dimasukkan tidak memenuhi syarat, bagian else akan menampilkan pesan kegagalan. Jadi method ini tidak hanya

mengubah saldo, tetapi juga memberikan respon kepada pengguna berdasarkan kondisi input.

g. Method cekInformasi

```
28
29⊖     public void cekInformasi() {
30          System.out.println("--- INFO REKENING ---");
31          System.out.println("No. Rekening : " + nomorRekening);
```

iMethod cekInformasi digunakan untuk menampilkan data rekening yang sedang digunakan. Program menampilkan nomor rekening sebagai identitas object. Method ini tidak mengubah saldo karena tugasnya hanya membaca dan menampilkan state yang sudah tersimpan.

h. Menampilkan nama dan saldo

```
32          System.out.println("Nama Pemilik : " + namaPemilik);
33          System.out.println("Saldo Akhir  : Rp" + saldo);
34          System.out.println("---------------------");
35      }
36  }
```

Tiga baris ini melanjutkan tampilan informasi rekening dengan menampilkan nama pemilik dan saldo terakhir. Garis pembatas digunakan agar hasil di console terlihat lebih rapi. Dari output ini dapat diketahui apakah perubahan saldo dari method setorTunai sudah tersimpan pada object yang sama.

2.3 Program Dasar - Class pekan1.Main

Class kedua adalah Main.java pada package pekan1. Class ini sesuai dengan bagian modul yang menjelaskan pembuatan Engine CLI. Main bertugas menerima input pengguna melalui Scanner, menampilkan menu, menjalankan perulangan, dan memanggil method yang ada pada object Rekening.

a. Package dan Scanner

```
1  package pekan1;
2
3  import java.util.Scanner;
4
5  public class Main {
```

Program Main ditempatkan pada package pekan1 agar berada pada kelompok yang sama dengan class Rekening dasar. Library Scanner diimpor dari java.util karena program membutuhkan input dari keyboard. Setelah itu dibuat class Main sebagai

pusat eksekusi program. Pembagian ini membuat fungsi menerima input tidak dicampur dengan fungsi penyimpanan data rekening.

b. Method main dan Scanner

```
public static void main(String[] args) {
    Scanner input = new Scanner(System.in);
    Rekening akunAktif = null;
    boolean isRunning = true;
```

Method main merupakan titik awal ketika program Java dijalankan. Object Scanner dibuat untuk membaca input pengguna melalui console. Variabel akunAktif awalnya bernilai null karena belum ada rekening yang dibuat. Variabel isRunning digunakan sebagai penanda agar perulangan menu dapat terus berjalan sampai pengguna memilih keluar.

c. Menampilkan judul dan perulangan menu

```
12          System.out.println("=== SISTEM PERBANKAN MINI ===");
13
14          while (isRunning) {
15
16              System.out.println("\nMenu Utama:");
```

Program terlebih dahulu menampilkan judul Mini Bank. Setelah itu while digunakan agar menu dapat muncul berulang kali tanpa harus menjalankan program dari awal.

Selama isRunning masih bernilai true, pengguna dapat memilih berbagai operasi yang tersedia. Perulangan akan berhenti ketika nilai isRunning diubah menjadi false pada pilihan keluar.

d. Menu utama

```
17              System.out.println("1. Buka Rekening Baru");
18              System.out.println("2. Setor Tunai");
19              System.out.println("3. Tarik Tunai");
20              System.out.println("4. Cek Informasi Rekening");
21              System.out.println("0. Keluar");
22              System.out.print("Pilih menu: ");
```

Bagian ini menampilkan pilihan menu yang dapat dipilih oleh pengguna. Pada program dasar, menu tarik tunai masih berupa bagian yang akan dikerjakan pada tugas mandiri, sesuai alur modul. Menu buka rekening digunakan untuk membuat

object, menu setor untuk memanipulasi saldo, dan menu cek informasi untuk melihat data rekening.

e. Membaca pilihan pengguna

```
24              int pilihan = input.nextInt();
25              input.nextLine();
26
27              switch (pilihan) {
```

Input menu dibaca menggunakan nextInt karena pilihan berbentuk angka. Setelah itu nextLine digunakan untuk membersihkan sisa enter sehingga pembacaan teks berikutnya tidak terganggu. Nilai pilihan kemudian diproses menggunakan switch.

Dengan struktur ini, setiap nomor menu dapat mempunyai proses yang berbeda.

f. Membuka rekening

```
29              case 1:
30                  System.out.print("Masukkan No Rekening: ");
31                  String no = input.nextLine();
32
33                  System.out.print("Masukkan Nama Pemilik: ");
34                  String nama = input.nextLine();
```

Pada case 1 program meminta nomor rekening dan nama pemilik. Kedua data tersebut disimpan sementara pada variabel no dan nama. Selanjutnya program juga meminta saldo awal. Data yang dikumpulkan ini akan digunakan untuk membuat object Rekening baru melalui constructor.

g. Membuat object Rekening

```
36                  System.out.print("Masukkan Saldo Awal: ");
37                  double saldo = input.nextDouble();
38
39                  akunAktif = new Rekening(no, nama, saldo);
40                  break;
```

Keyword new digunakan untuk membuat object Rekening berdasarkan data yang dimasukkan pengguna. Object tersebut kemudian disimpan pada variabel akunAktif.

Karena program dasar hanya mempunyai satu variabel akun aktif, rekening baru akan menjadi rekening yang sedang digunakan. Proses ini merupakan contoh hubungan antara class, object, dan constructor.

h. Menu setor tunai

```
41
42            case 2:
43                if (akunAktif == null) {
44                    System.out.println(
45                            "Error: Mohon maaf, Anda belum memiliki nomor rekening!");
```

Sebelum melakukan setor tunai, program mengecek apakah akunAktif masih null.

Jika belum ada rekening, pengguna diberi pesan bahwa rekening belum dimiliki.

Pengecekan ini penting supaya method setorTunai tidak dipanggil pada object yang belum ada. Dengan demikian program lebih aman saat menerima berbagai kondisi input pengguna.

i. Memanggil method setorTunai

```
46                } else {
47                    System.out.print("Masukkan nominal setor: ");
48                    double setor = input.nextDouble();
49
50                    akunAktif.setorTunai(setor);
51                }
52                break;
```

Jika akun aktif sudah tersedia, program meminta nominal setor dari pengguna. Nilai tersebut kemudian dikirim sebagai parameter ke method setorTunai milik akunAktif.

Perubahan saldo dilakukan di dalam class Rekening, sedangkan Main hanya mengatur interaksi dan memanggil method. Pembagian tugas seperti ini membuat struktur program lebih mudah dipahami.

j. Menu tarik tunai pada program dasar

```
case 3:
    System.out.println( "Fitur tarik tunai akan dikerjakan sebagai Tugas Mandiri.");
    break;
```

Pada program dasar, menu nomor 3 belum melakukan penarikan saldo. Pesan tersebut menunjukkan bahwa fitur tarik tunai memang menjadi bagian latihan atau tugas

sesuai modul. Karena itu, bagian ini kemudian dikembangkan pada package TugasPekan1. Perubahan tersebut dibahas pada bagian berikutnya.

k. Cek informasi dan keluar

```
case 4:
    if (akunAktif == null) {
        System.out.println( "Error: Anda belum membuka rekening!");
    } else {
        akunAktif.cekInformasi();
    }
    break;
```

Menu nomor 4 digunakan untuk melihat informasi rekening. Sama seperti setor tunai, program terlebih dahulu memastikan akunAktif tidak null. Jika sudah ada rekening, method cekInformasi dipanggil sehingga data nomor rekening, nama, dan saldo tampil di console.

l. Mengakhiri program

```
case 0:
    isRunning = false;
    System.out.println("Sistem ditutup. Terima kasih!");
    break;
```

Ketika pengguna memilih angka 0, nilai isRunning diubah menjadi false. Akibatnya kondisi while tidak lagi terpenuhi dan perulangan berhenti. Program kemudian menampilkan pesan penutup. Scanner selanjutnya ditutup agar resource input tidak dibiarkan terbuka.

2.4 Tugas Praktikum - Class TugasPekan1.Rekening

Setelah program dasar selesai, bagian tugas dikembangkan sesuai instruksi pada modul.

Tugas pertama adalah implementasi fitur tarik tunai dengan aturan saldo tidak boleh minus dan minimal penarikan Rp10.000. Oleh karena itu class Rekening pada package TugasPekan1 mempunyai method tambahan tarikTunai selain method yang sudah ada pada program dasar.

a. Package dan class tugas

```java
package TugasPekan1;

public class Rekening {

    String nomorRekening;
    String namaPemilik;
    double saldo;
```

Class Rekening pada bagian tugas dibuat dalam package TugasPekan1 agar terpisah dari program dasar. Atribut yang digunakan tetap sama, yaitu nomor rekening, nama pemilik, dan saldo. Pemisahan package ini membantu menunjukkan bahwa class tersebut merupakan versi yang sudah dikembangkan untuk memenuhi tugas.

b. Constructor pada class tugas

```java
    public Rekening(String nomor, String nama, double saldoAwal) {
        nomorRekening = nomor;
        namaPemilik = nama;
        saldo = saldoAwal;
```

Constructor tetap digunakan untuk memberikan state awal object. Nilai nomor rekening, nama pemilik, dan saldo awal diambil dari parameter yang diberikan ketika object dibuat. Dengan begitu, setiap rekening yang dibuat pada program tugas mempunyai identitas dan saldo awal sendiri.

c. Method setor tunai tetap digunakan

```java
    public void setorTunai(double nominal) {
        if (nominal > 0) {
            saldo += nominal;
```

Fitur setor tunai tetap dipertahankan pada versi tugas karena fitur tersebut merupakan bagian dari program dasar. Kondisi nominal lebih dari nol tetap digunakan agar saldo tidak bertambah akibat input nol atau negatif. Jika setor berhasil, state saldo object akan berubah sesuai nominal yang dimasukkan.

d. Awal method tarikTunai

```
27⊖     public void tarikTunai(double nominal) {
28
29⊖         if (nominal > saldo) {
30              System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp" + saldo);
```

Method tarikTunai merupakan fitur baru yang diminta pada tugas. Kondisi pertama mengecek apakah nominal penarikan lebih besar daripada saldo. Jika iya, transaksi

tidak boleh dilakukan karena dapat menyebabkan saldo menjadi minus. Program justru menampilkan peringatan bahwa saldo tidak mencukupi, sesuai aturan yang diberikan pada modul.

e. Aturan minimal penarikan

```
        } else if (nominal < 10000) {

            System.out.println(
                    "Transaksi Gagal : Minimal nomonal penarikan 10.000");
```

Setelah saldo mencukupi, program memeriksa syarat kedua, yaitu nominal penarikan minimal Rp10.000. Jika nominal lebih kecil dari nilai tersebut, saldo tidak dikurangi.

Program menampilkan pesan peringatan sesuai format pada modul. Dengan adanya dua pemeriksaan ini, transaksi yang tidak memenuhi aturan dapat ditolak tanpa membuat program crash.

f. Penarikan berhasil

```
        } else {
            saldo -= nominal;
            System.out.println("Tarik tunai Rp" + nominal+ " berhasil. Saldo saat ini: Rp" + saldo);
        }
    }
```

Jika nominal tidak lebih besar dari saldo dan tidak kurang dari Rp10.000, transaksi dianggap valid. Nilai saldo kemudian dikurangi dengan nominal penarikan. Setelah itu program menampilkan nominal yang ditarik dan saldo terbaru. Bagian ini menunjukkan manipulasi state object karena nilai atribut saldo berubah setelah method dijalankan.

g. Method cekInformasi pada tugas

```
42      public void cekInformasi() {
43          System.out.println("--- INFO REKENING ---");
44          System.out.println("No. Rekening : " + nomorRekening);
45          System.out.println("Nama Pemilik : " + namaPemilik);
46          System.out.println("Saldo Akhir  : Rp" + saldo);
47          System.out.println("----------------------");
48      }
49  }
```

Method cekInformasi pada class tugas tetap digunakan untuk melihat keadaan rekening setelah transaksi. Ini berguna saat pengujian karena saldo dapat

dibandingkan sebelum dan sesudah setor atau tarik tunai. Dengan melihat saldo akhir, pengguna dapat memastikan bahwa perubahan state sudah terjadi sesuai transaksi yang berhasil.

2.5 Tugas Praktikum - Class TugasPekan1.Main

Tugas kedua pada modul adalah Tantangan Multi-Akun. Program dasar hanya mempunyai satu variabel akunAktif sehingga rekening lama dapat tertimpa ketika rekening baru dibuat.

Pada versi tugas, penyimpanan diubah menggunakan ArrayList<Rekening> sehingga beberapa object Rekening dapat disimpan sekaligus. Selain itu, ditambahkan menu Ganti Akun untuk mencari rekening berdasarkan nomor rekening.

a. Import ArrayList dan Scanner

```
1  package TugasPekan1;
2
3  import java.util.ArrayList;
5
6  public class Main {
```

Pada Main versi tugas terdapat dua library yang digunakan. Scanner tetap diperlukan untuk menerima input dari pengguna, sedangkan ArrayList digunakan sebagai collection untuk menyimpan banyak object Rekening. Ini merupakan perubahan penting dibandingkan program dasar yang hanya menyimpan satu object melalui akunAktif.

b. Menyiapkan daftar rekening

```
Scanner input = new Scanner(System.in);

ArrayList<Rekening> daftarRekening =new ArrayList<Rekening>();
Rekening akunAktif = null;
boolean isRunning = true;
```

Variabel daftarRekening digunakan sebagai tempat menyimpan seluruh rekening

yang berhasil dibuat. Tipe datanya ArrayList<Rekening>, artinya isi collection berupa object dari class Rekening. akunAktif tetap dipertahankan untuk menunjukkan rekening mana yang sedang digunakan pada saat itu. Jadi, daftarRekening

menyimpan semua akun, sedangkan akunAktif menunjuk satu akun yang sedang aktif.

c. Menu yang sudah dikembangkan

```
20              System.out.println("\nMenu Utama:");
21              System.out.println("1. Buka Rekening Baru");
22              System.out.println("2. Setor Tunai");
23              System.out.println("3. Tarik Tunai");
24              System.out.println("4. Cek Informasi Rekening");
25              System.out.println("5. Ganti Akun");
26              System.out.println("6. Keluar");
27              System.out.print("Pilih menu: ");
```

Menu versi tugas mempunyai tambahan pilihan nomor 5 yaitu Ganti Akun. Pilihan nomor 3 sekarang sudah terhubung dengan method tarikTunai. Pilihan Ganti Akun diperlukan karena program dapat menyimpan lebih dari satu rekening, sehingga pengguna perlu cara untuk memilih rekening lain yang sudah tersimpan.

d. Membuat rekening baru dan memasukkannya ke ArrayList

```
Rekening rekeningBaru =new Rekening(no, nama, saldo);
daftarRekening.add(rekeningBaru);
akunAktif = rekeningBaru;
```

Saat pengguna membuka rekening baru, object terlebih dahulu dibuat menggunakan constructor Rekening. Object tersebut kemudian ditambahkan ke daftarRekening dengan method add. Setelah disimpan, object yang sama dijadikan akunAktif sehingga rekening yang baru dibuat langsung menjadi akun yang sedang digunakan.

Dengan cara ini rekening lama tetap berada di ArrayList dan tidak hilang.

e. Menu setor tunai pada akun aktif

```
50
51            case 2:
52
53                if (akunAktif == null) {
54
55                    System.out.println(
56                            "Error: Mohon maaf, Anda belum memiliki nomor rekening!");
57
58                } else {
59
60                    System.out.print("Masukkan nominal setor: ");
```

Pada menu setor tunai, pengecekan akun aktif tetap dilakukan. Hal ini penting karena pengguna bisa saja memilih setor sebelum membuka rekening. Jika akun aktif

tersedia, nominal setor dibaca dari input dan kemudian dikirim ke method setorTunai. Method pada class Rekening yang bertugas mengubah saldo, bukan Main.

f. Menu tarik tunai

```
68            case 3:
69
70                if (akunAktif == null) {
71
72                    System.out.println("Error: Mohon maaf, Anda belum memiliki nomor rekening!");
73
74                } else {
75
76                    System.out.print("Masukkan nominal tarik: ");
```

Menu nomor 3 sekarang sudah menjalankan fitur yang diminta pada tugas. Program tetap mengecek akun aktif sebelum meminta nominal penarikan. Jika akun tersedia, input nominal akan diteruskan ke method tarikTunai. Aturan saldo dan minimal penarikan kemudian diproses di class Rekening.

g. Memanggil tarikTunai

```
76                    double tarik = input.nextDouble();
77
78                    akunAktif.tarikTunai(tarik);
79                }
```

Nilai yang dimasukkan pengguna disimpan pada variabel tarik. Selanjutnya method tarikTunai dipanggil menggunakan akunAktif sebagai object yang menjalankan method. Jika transaksi berhasil, saldo akun aktif berkurang. Jika transaksi tidak memenuhi aturan, method hanya menampilkan pesan gagal tanpa mengubah saldo.

h. Menu ganti akun

```
96            case 5:
97
98                System.out.print("Masukkan nomor rekening: ");
99                String nomorCari = input.nextLine();
100
101               boolean ditemukan = false;
```

Menu Ganti Akun meminta nomor rekening yang ingin digunakan. Variabel nomorCari menyimpan nomor yang dimasukkan pengguna. Variabel ditemukan

digunakan untuk menandai apakah rekening yang dicari benar-benar ada di dalam ArrayList. Nilai awalnya false karena pencarian belum dilakukan.

i. Mencari rekening dengan for-each

```
for (Rekening rekening : daftarRekening) {

    if (rekening.nomorRekening.equals(nomorCari)) {

        akunAktif = rekening;
        ditemukan = true;
```

Perulangan for-each digunakan untuk memeriksa object Rekening satu per satu di dalam daftarRekening. Nomor rekening setiap object dibandingkan dengan nomor yang dimasukkan pengguna menggunakan equals. Jika ditemukan kecocokan, object tersebut dijadikan akunAktif. Dengan demikian pengguna dapat berpindah dari satu rekening ke rekening lainnya tanpa menghapus akun sebelumnya.

j. Pesan hasil pencarian akun

```
            System.out.println( "Akun berhasil diganti.");

            break;
        }
    }

    if (!ditemukan) {
        System.out.println("Rekening tidak ditemukan.");
    }
```

Jika nomor rekening cocok, program menampilkan pesan bahwa akun berhasil diganti dan perulangan dihentikan. Jika seluruh isi ArrayList sudah diperiksa tetapi tidak ada yang cocok, variabel ditemukan tetap false. Kondisi tersebut membuat program menampilkan pesan bahwa rekening tidak ditemukan. Cara ini membuat fitur Ganti Akun tetap aman ketika pengguna memasukkan nomor yang tidak terdaftar.

k. Keluar dan menutup Scanner

```
            case 0:
                isRunning = false;
                System.out.println ( "Sistem ditutup. Terima kasih!");
                break;
            default:
                System.out.println("Pilihan tidak valid!");
            }
        }
        input.close();
```

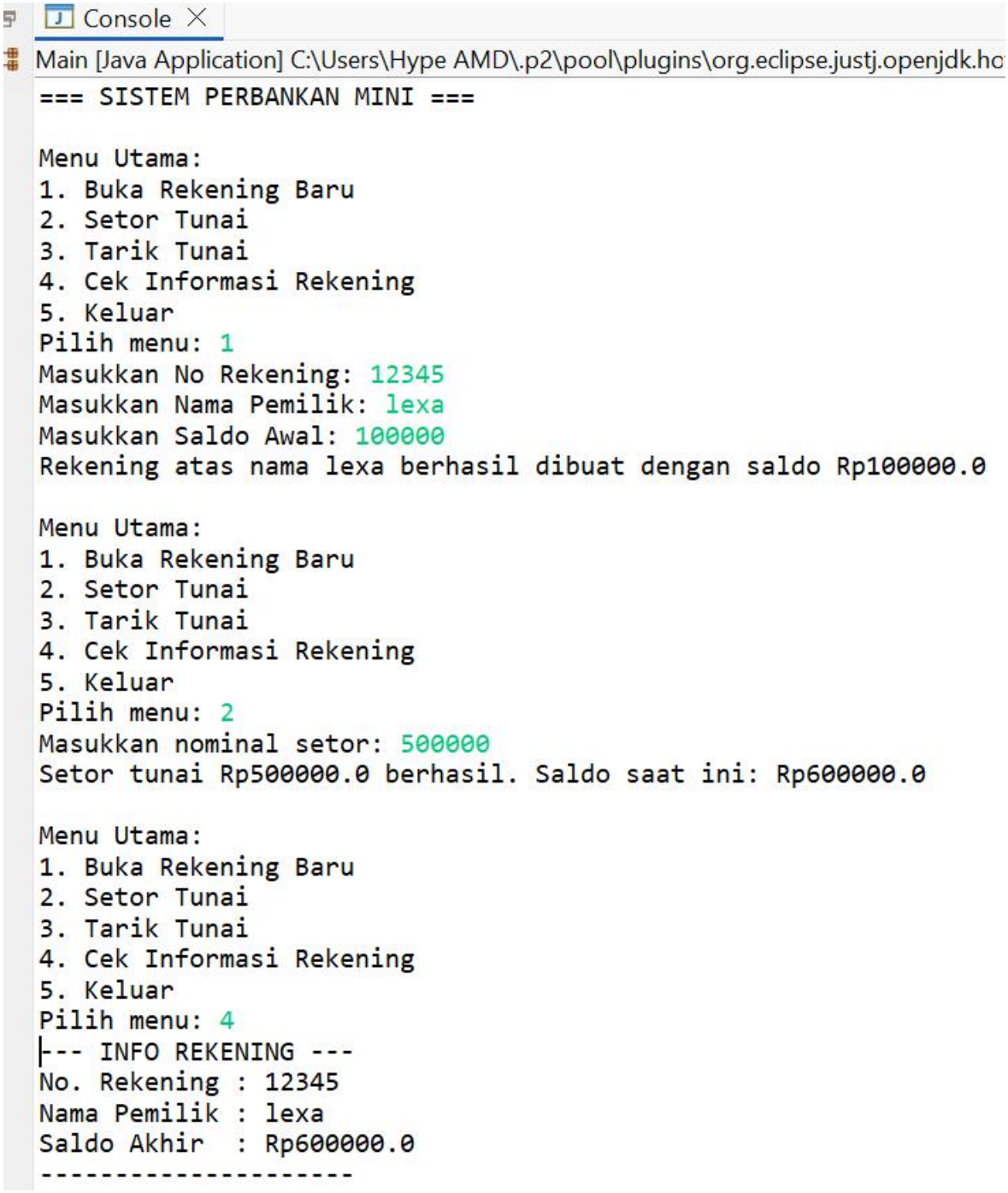
Pilihan 0 menghentikan perulangan dengan mengubah isRunning menjadi false.

Setelah keluar dari perulangan, Scanner ditutup menggunakan input.close().

Penutupan Scanner dilakukan pada akhir program agar resource input ditutup dengan benar. Bagian ini sekaligus menandai bahwa proses program sudah selesai.

2.6 Pengujian dan Hasil Praktikum

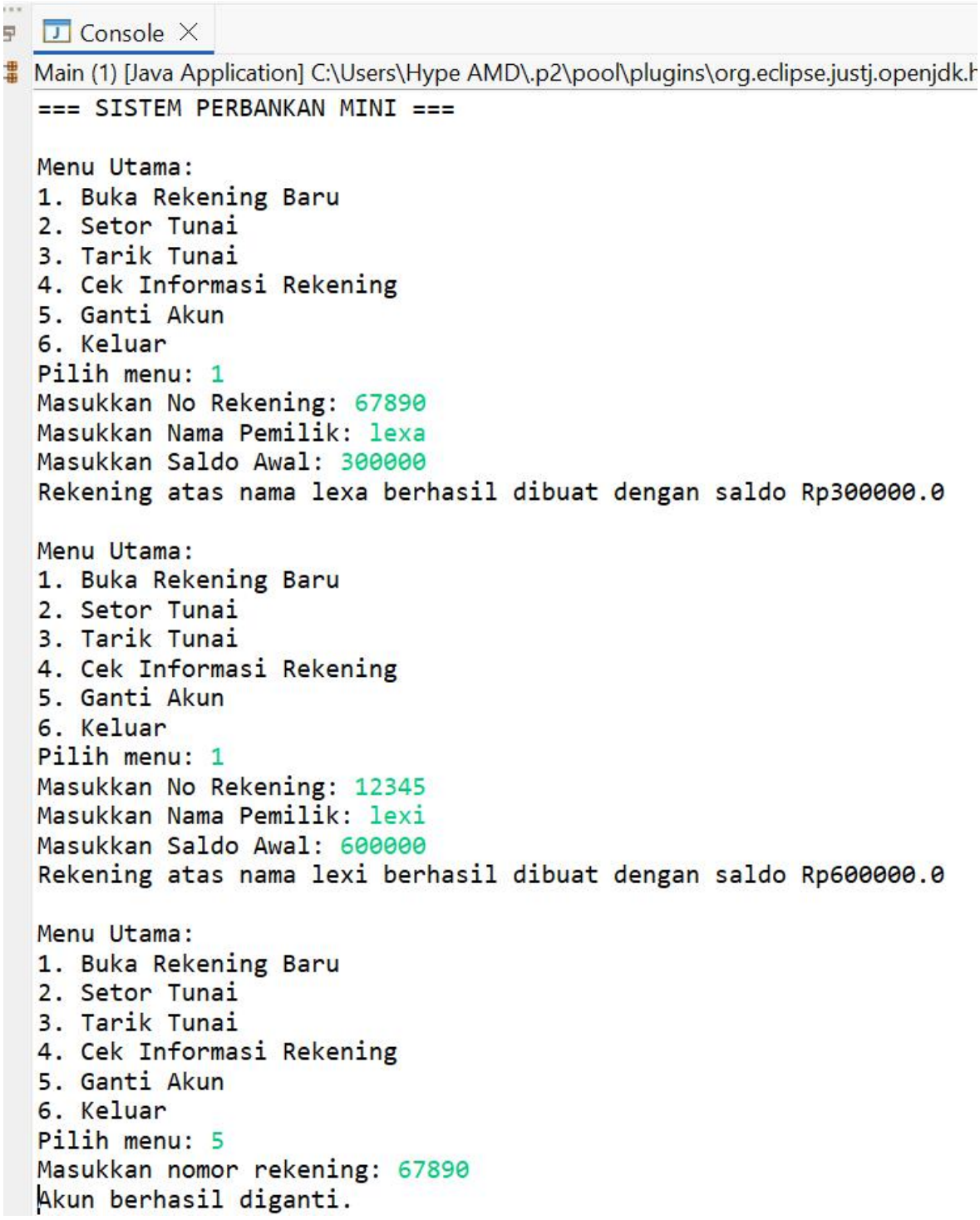
Setelah empat class selesai dibuat, program diuji melalui console. Pengujian dilakukan pada program dasar untuk memastikan pembukaan rekening, setor tunai, dan pengecekan informasi berjalan. Program tugas kemudian diuji untuk memastikan tarik tunai, penolakan nominal di bawah Rp10.000, penolakan ketika saldo tidak mencukupi, pembuatan lebih dari satu rekening, dan Ganti Akun dapat berjalan.



Gambar 2.5 Hasil pengujian program dasar

Berdasarkan hasil praktikum pada Console Eclipse, program Sistem Perbankan Mini berhasil dijalankan dengan baik. Pada percobaan pertama, pengguna memilih menu 1. Buka Rekening Baru, kemudian memasukkan nomor rekening 12345, nama pemilik

lexa, dan saldo awal Rp100000. Setelah rekening berhasil dibuat, pengguna memilih menu 2. Setor Tunai dan memasukkan nominal Rp500000, sehingga saldo bertambah menjadi Rp600000. Selanjutnya dipilih menu 4. Cek Informasi Rekening, dan program menampilkan nomor rekening, nama pemilik, serta saldo akhir sebesar Rp600000. Hasil tersebut menunjukkan bahwa proses pembuatan rekening, setor tunai, dan pengecekan informasi rekening dapat berjalan sesuai dengan program yang dibuat.



Gambar 2.6 Hasil pengujian fitur tugas

Pada pengujian ini, program digunakan untuk membuat dua rekening. Rekening pertama dibuat dengan nomor 67890, nama pemilik lexa, dan saldo awal Rp300000,

kemudian rekening kedua dibuat dengan nomor 12345, nama pemilik lexi, dan saldo awal Rp600000.

Setelah kedua rekening berhasil dibuat, pengguna memilih menu 5. Ganti Akun dan memasukkan nomor rekening 67890. Program kemudian menampilkan pesan “Akun berhasil diganti.”, yang menunjukkan bahwa fitur ganti akun berhasil mencari rekening berdasarkan nomor rekening dan menjadikan rekening tersebut sebagai akun yang aktif.

PRAKTIKUM 2

RELASI ANTAR OBJEK (CLASS DIAGRAM & ASOSIASI)

PEMBAHASAN

A. LATIHAN

2.1 Persiapan Project dan Package

Praktikum 2 merupakan lanjutan dari program pada praktikum sebelumnya sehingga project yang digunakan tetap PrakPBO_B_2511531006. Pada source folder dibuat package baru bernama pekan2. Package tersebut digunakan untuk menyimpan class Transaksi, Rekening, dan Main pada Praktikum 2.

a. Package pada class

```
1 package pekan2;
2
```

Baris package digunakan untuk menentukan tempat class dalam struktur project.

Dengan menggunakan package pekan2, seluruh class Praktikum 2 dapat dikelompokkan dengan rapi dan tidak bercampur dengan class pada praktikum sebelumnya. Package juga membantu Java membedakan class yang mempunyai nama sama tetapi berada pada package berbeda.Pada package pekan2 terdapat tiga class utama. Transaksi bertugas menyimpan detail transaksi, Rekening menyimpan data rekening sekaligus riwayat transaksi, sedangkan Main mengatur input pengguna dan menu program. Pembagian tersebut membuat setiap class mempunyai fungsi yang lebih jelas.

2.2 Class Transaksi

Class Transaksi merupakan class baru pada Praktikum 2. Class ini digunakan untuk mewakili satu transaksi yang terjadi pada rekening. Data yang disimpan terdiri dari ID transaksi, jenis transaksi, dan nominal. Object Transaksi nantinya dimasukkan ke dalam ArrayList riwayatTransaksi milik class Rekening.

a. Package dan deklarasi class

```
1  package pekan2;
2
3  public class Transaksi {
4
```

Baris package menempatkan class Transaksi pada package pekan2. Setelah itu public class Transaksi mendeklarasikan class yang dapat digunakan oleh class lain.

Tanda kurung kurawal membuka isi class sehingga atribut, constructor, dan method dapat didefinisikan di dalamnya.

b. Atribut transaksi

```
5      String idTransaksi;
6      String jenis;
7      double nominal;
```

Ketiga baris tersebut merupakan atribut yang menyimpan state dari object Transaksi. idTransaksi digunakan sebagai identitas transaksi, jenis menyimpan keterangan Kredit atau Debit, sedangkan nominal menyimpan jumlah uang yang terlibat. Tipe double dipakai untuk menyimpan nilai nominal.

c. Constructor Transaksi

```
8
9      public Transaksi(String idTransaksi, String jenis, double nominal) {
10         this.idTransaksi = idTransaksi;
11         this.jenis = jenis;
12         this.nominal = nominal;
13     }
```

Constructor digunakan untuk memberikan nilai awal ketika object Transaksi dibuat menggunakan keyword new. Parameter constructor menerima ID, jenis, dan nominal dari program yang membuat object. Keyword this digunakan untuk menunjukkan atribut milik object, sehingga parameter dan atribut yang mempunyai nama sama tetap dapat dibedakan.

d. Method cetakDetail()

```
    }

    public void cetakDetail() {
        System.out.println("ID Transaksi : " + idTransaksi+ " | Jenis : " + jenis + " | Nominal : Rp" + nominal);
    }
}
```

Method cetakDetail() digunakan untuk menampilkan isi sebuah object Transaksi ke Console. System.out.println menggabungkan teks dengan nilai atribut menggunakan operator +. Ketika method ini dipanggil, ID, jenis, dan nominal transaksi akan ditampilkan dalam satu baris agar mudah dibaca.

2.3 Relasi Class Rekening dan Transaksi

Relasi antara Rekening dan Transaksi dibuat karena satu rekening dapat memiliki banyak transaksi. Hubungan tersebut diterapkan dengan membuat atribut ArrayList<Transaksi> pada class Rekening. Dengan demikian setiap object Rekening mempunyai tempat sendiri untuk menyimpan kumpulan transaksi yang dilakukan.

a. Import ArrayList

```
2
3  import java.util.ArrayList;
```

Baris ini mengimpor class ArrayList dari library Java. ArrayList digunakan karena jumlah transaksi dalam satu rekening dapat bertambah selama program berjalan.

Dengan ArrayList, object Transaksi dapat disimpan dan ditambahkan satu per satu.

b. Deklarasi class dan atribut rekening

```
public class Rekening {
    String nomorRekening;
    String namaPemilik;
    double saldo;
```

Baris pertama mendeklarasikan class Rekening. Tiga atribut berikutnya menyimpan nomor rekening, nama pemilik, dan saldo. Ketiga data tersebut merupakan state utama yang dimiliki oleh setiap object Rekening.

c. Membuat riwayat transaksi

```
9
10      ArrayList<Transaksi> riwayatTransaksi =new ArrayList<Transaksi>();
11
```

Baris ini membuat ArrayList yang hanya menerima object bertipe Transaksi.

Variabel riwayatTransaksi menjadi tempat penyimpanan seluruh transaksi milik rekening. Inilah bagian utama yang menunjukkan relasi antara object Rekening dengan banyak object Transaksi.

d. Constructor Rekening

```
12      public Rekening(String nomor, String nama, double saldoAwal) {
13          nomorRekening = nomor;
14          namaPemilik = nama;
15          saldo = saldoAwal;
```

Constructor Rekening menerima nomor, nama, dan saldo awal. Nilai tersebut kemudian disimpan ke atribut object agar setiap rekening mempunyai data awal yang berbeda. Constructor dijalankan saat object Rekening dibuat dengan keyword new.

2.4 Pencatatan Setor Tunai

Pada latihan ini method setorTunai dikembangkan agar transaksi yang berhasil tidak hanya mengubah saldo, tetapi juga dicatat sebagai object Transaksi. Jenis transaksi setor diberi nilai Kredit dan ID transaksi menggunakan awalan TRX-S-. Jika nominal tidak memenuhi syarat, program hanya menampilkan pesan gagal.

a. Method dan validasi

```
19
20      public void setorTunai(double nominal) {
```

Method setorTunai menerima satu parameter nominal. Kondisi if memeriksa apakah nominal lebih besar dari nol. Jika kondisi benar, proses setoran dilanjutkan;

jika tidak, program masuk ke bagian else dan transaksi tidak dicatat.

b. Menambah saldo

```
saldo += nominal;
```

Operator += menambahkan nilai nominal ke saldo yang sudah ada. Baris ini mengubah state saldo pada object Rekening. Perubahan saldo dilakukan terlebih dahulu sebelum transaksi berhasil dibuat dan disimpan.

c. Membuat transaksi Kredit

```
Transaksi transaksiBaru =new Transaksi("TRX-S-" + (riwayatTransaksi.size() + 1),"Kredit",nominal);
riwayatTransaksi.add(transaksiBaru);
System.out.println("Setor tunai Rp" + nominal+" berhasil. Saldo saat ini: Rp" + saldo);
```

Setelah saldo berhasil ditambah, program membuat object Transaksi baru. ID

diberi awalan TRX-S-, jenis transaksi diisi Kredit, dan nominal menggunakan nilai setoran. Object tersebut mewakili satu transaksi setor yang berhasil.

d. Menyimpan transaksi

```
riwayatTransaksi.add(transaksiBaru);
```

Method add memasukkan object transaksiBaru ke dalam ArrayList. Setelah baris ini dijalankan, transaksi setor sudah menjadi bagian dari riwayat rekening dan dapat ditampilkan kembali melalui fitur cetak mutasi.

e. Kondisi gagal

```
} else {
    System.out.println("Gagal: Nominal setor harus lebih dari 0!");
}
```

Bagian else dijalankan jika nominal tidak lebih besar dari nol. Program menampilkan pesan bahwa nominal setor harus lebih dari 0. Karena tidak ada object Transaksi yang dibuat pada bagian ini, setoran gagal tidak masuk ke riwayat.

B. TUGAS

2.5 Tugas 1 – Integrasi Rekam Jejak Penarikan

Tugas 1 meminta agar setiap penarikan yang berhasil mempunyai rekam jejak transaksi. Program juga harus memastikan transaksi gagal tidak dicatat. Oleh karena itu, object Transaksi Debit dibuat hanya setelah pemeriksaan saldo dan batas minimum penarikan berhasil dilewati.

a. Method tarikTunai dan pemeriksaan saldo

```
34    public void tarikTunai(double nominal) {
35
36        if (nominal > saldo) {
37
38            System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp"+ saldo);
```

Method tarikTunai menerima nominal penarikan dari pengguna. Kondisi pertama memeriksa apakah nominal lebih besar daripada saldo yang tersedia. Jika

benar, transaksi dihentikan dan hanya pesan kegagalan yang ditampilkan sehingga saldo dan riwayat tidak berubah.

b. Pemeriksaan minimum penarikan

```
10⊖        } else if (nominal < 10000) {
11
12             System.out.println("Transaksi Gagal : Minimal nominal penarikan 10.000");
```

Jika saldo mencukupi, program memeriksa syarat minimum penarikan. Nominal kurang dari Rp10.000 dianggap tidak memenuhi aturan program. Pada kondisi ini saldo tidak dikurangi dan transaksi juga tidak dimasukkan ke riwayat.

c. Penarikan berhasil

```
43
44⊖        } else {
45
46             saldo -= nominal;
```

Blok else hanya dijalankan jika saldo mencukupi dan nominal tidak kurang dari Rp10.000. Operator -= mengurangi saldo sesuai jumlah uang yang ditarik. Setelah saldo berubah, program melanjutkan pencatatan transaksi.

d. Membuat transaksi Debit

```
Transaksi transaksiBaru =new Transaksi("TRX-T-" + (riwayatTransaksi.size() + 1),"Debit",nominal);
```

Object Transaksi dibuat untuk merekam penarikan yang berhasil. ID menggunakan awalan TRX-T-, jenis transaksi diisi Debit, dan nominal berisi jumlah penarikan.

Posisi kode ini penting karena object hanya dibuat setelah penarikan dinyatakan berhasil.

e. Menyimpan dan menampilkan transaksi

```
            riwayatTransaksi.add(transaksiBaru);
            System.out.println("Tarik tunai Rp" + nominal+" berhasil. Saldo saat ini: Rp" + saldo);
        }
    }
```

Object transaksiBaru ditambahkan ke riwayat sehingga penarikan berhasil dapat dilihat kembali. Setelah itu program menampilkan pesan keberhasilan dan saldo terbaru. Penarikan yang gagal tidak mencapai bagian add sehingga tidak dicatat sebagai transaksi berhasil.

2.6 Tugas 2 – Fitur Cetak Mutasi Rekening

Tugas 2 menambahkan method cetakMutasi pada class Rekening. Method ini bertugas membaca seluruh object Transaksi yang tersimpan dalam ArrayList dan menampilkannya.

Jika belum ada transaksi, program menampilkan pesan khusus sesuai ketentuan tugas.

a. Method cetakMutasi

```
62    public void cetakMutasi() {
63
64        System.out.println("==MUTASI REKENING==");
```

Method cetakMutasi dibuat pada class Rekening karena riwayat transaksi merupakan bagian dari data rekening. Perintah println pertama menampilkan judul agar hasil mutasi mudah dibedakan dari output menu atau transaksi lainnya.

b. Memeriksa riwayat kosong

```
66        if (riwayatTransaksi.isEmpty()) {
67
68            System.out.println("Belum ada transaksi pada rekening ini");
```

Method isEmpty() digunakan untuk mengetahui apakah ArrayList tidak memiliki data. Jika kosong, program menampilkan pesan bahwa belum ada transaksi.

Kondisi ini mencegah program melakukan perulangan terhadap data yang memang belum tersedia.

c. Menampilkan seluruh transaksi

```
69
70          } else {
71
72              for (Transaksi transaksi : riwayatTransaksi) {
73
74                  transaksi.cetakDetail();
75              }
76          }
```

Jika riwayat tidak kosong, program menggunakan for-each untuk mengambil setiap object Transaksi. Variabel transaksi mewakili satu object yang sedang diproses. Method cetakDetail kemudian dipanggil sehingga setiap transaksi ditampilkan satu per satu.

d. Penutup output

```
78              System.out.println("-----------------------");
79      }
80  }
```

Garis pembatas digunakan untuk membuat tampilan Console lebih rapi. Kurung kurawal menutup method cetakMutasi setelah seluruh proses pemeriksaan dan pencetakan selesai.

2.7 Penambahan Menu 6 pada Main

Agar fitur cetak mutasi dapat digunakan pengguna, menu utama pada class Main ditambah pilihan nomor 6. Ketika pengguna memilih nomor tersebut, program akan memanggil cetakMutasi pada akun yang sedang aktif.

a. Menambahkan menu

```
24              System.out.println("5. Ganti Akun");
25              System.out.println("6. Cetak Mutasi (Riwayat)");
26              System.out.println("7. Keluar");
```

Ketiga baris tersebut menampilkan pilihan menu pada Console. Menu nomor 6 merupakan fitur baru untuk melihat riwayat transaksi, sedangkan nomor 0 tetap digunakan untuk mengakhiri program.

b. Case 6

```
case 6:
    if (akunAktif == null) {
        System.out.println("Error: Anda belum membuka rekening!");
```

case 6 dijalankan ketika pilihan pengguna bernilai 6. Sebelum mencetak mutasi, program memeriksa apakah akunAktif masih null. Jika belum ada rekening aktif, program menampilkan pesan kesalahan.

c. Memanggil cetakMutasi

```
    } else {
        akunAktif.cetakMutasi();
    }
    break;
```

Jika akun aktif tersedia, program memanggil method cetakMutasi pada object akunAktif. Dengan demikian data riwayat yang dicetak adalah milik rekening yang sedang dipilih. break menghentikan case 6 dan mengembalikan program ke menu utama.

2.8 Integrasi Program pada Class Main

Class Main menjadi penghubung seluruh class dalam program. Main menerima input pengguna, membuat rekening, memanggil method setor dan tarik, mengganti akun, serta menjalankan fitur cetak mutasi. Bagian ini menunjukkan bagaimana object Rekening dan Transaksi digunakan bersama dalam satu alur program.

a. Package dan import

```
1  package pekan2;
2
3  import java.util.ArrayList;
4  import java.util.Scanner;
```

Package menentukan lokasi class Main. ArrayList diimpor untuk menyimpan daftar rekening, sedangkan Scanner digunakan untuk menerima input dari keyboard. Kedua library tersebut diperlukan agar menu dan penyimpanan beberapa rekening dapat berjalan.

b. Deklarasi class dan main

```
public class Main {
    public static void main(String[] args) {
```

Baris pertama membuat class Main. Method main merupakan titik awal ketika program Java dijalankan. Parameter String[] args disediakan oleh Java untuk menerima argumen ketika program dijalankan dari lingkungan tertentu.

c. Membuat Scanner

```
Scanner input = new Scanner(System.in);
```

Object Scanner bernama input dibuat untuk membaca masukan pengguna.

System.in menunjukkan bahwa sumber input berasal dari keyboard melalui

console. Object ini digunakan pada proses input nomor rekening, nama, saldo, nominal setor, dan nominal tarik.

d. Daftar rekening dan akun aktif

```
ArrayList<Rekening> daftarRekening =new ArrayList<Rekening>();
Rekening akunAktif = null;
boolean isRunning = true;
```

daftarRekening menyimpan seluruh object Rekening yang dibuat pengguna.

akunAktif menyimpan rekening yang sedang digunakan, sedangkan nilai null berarti belum ada akun aktif. Variabel isRunning digunakan sebagai pengendali perulangan menu.

e. Perulangan menu

```
14
15⊖         while (isRunning) {
```

While menjalankan menu selama isRunning bernilai true. Selama pengguna belum memilih menu keluar, program akan kembali menampilkan pilihan dan menerima input baru. Ketika isRunning diubah menjadi false, perulangan berhenti.

f. Membaca pilihan menu

```
26
27              int pilihan = input.nextInt();
28              input.nextLine();
29
30⊖             switch (pilihan) {
```

nextInt membaca angka menu dan menyimpannya pada variabel pilihan. nextLine dipanggil setelahnya untuk membersihkan sisa enter sebelum program menerima input teks. Switch kemudian memilih proses berdasarkan nilai pilihan.

g. Case 1 membuka rekening

```
32⊖             case 1:
33
34                  System.out.print("Masukkan No Rekening: ");
35                  String no = input.nextLine();
36
37                  System.out.print("Masukkan Nama Pemilik: ");
38                  String nama = input.nextLine();
39
40                  System.out.print("Masukkan Saldo Awal: ");
41                  double saldo = input.nextDouble();
```

Case 1 meminta tiga data awal, yaitu nomor rekening, nama pemilik, dan saldo.

nextLine digunakan untuk membaca data berbentuk teks, sedangkan nextDouble digunakan untuk membaca saldo sebagai angka. Data tersebut kemudian dipakai untuk membuat object Rekening.

h. Membuat dan menyimpan rekening

```
Rekening rekeningBaru =new Rekening(no, nama, saldo);
daftarRekening.add(rekeningBaru);
akunAktif = rekeningBaru;
```

Object Rekening dibuat menggunakan constructor dengan data yang telah dimasukkan. Setelah dibuat, object ditambahkan ke daftarRekening menggunakan add. Object yang sama kemudian dijadikan akunAktif sehingga transaksi berikutnya menggunakan rekening tersebut.

i. Case 2 setor tunai

```
49          case 2:
50              if (akunAktif == null) {
51                  System.out.println("Error: Mohon maaf, Anda belum memiliki nomor rekening!");
52
53              } else {
54                  System.out.print("Masukkan nominal setor: ");
55                  double setor = input.nextDouble();
56
57                  akunAktif.setorTunai(setor);
58              }
59              break;
```

Case 2 digunakan untuk setor tunai. Program lebih dulu memastikan akun aktif tidak null. Jika tersedia, nominal dibaca menggunakan nextDouble dan method setorTunai dipanggil pada akun aktif. Setelah case selesai, break mengembalikan program ke menu.

j. Case 3 tarik tunai

```
61          case 3:
62              if (akunAktif == null) {
63                  System.out.println("Error: Mohon maaf, Anda belum memiliki nomor rekening!");
64              } else {
65                 System.out.print("Masukkan nominal tarik: ");
66                  double tarik = input.nextDouble();
67
68                  akunAktif.tarikTunai(tarik);
69              }
70
71              break;
```

Case 3 menjalankan proses tarik tunai. Pemeriksaan akun aktif dilakukan agar method tidak dipanggil pada object yang belum ada. Setelah nominal dibaca, method tarikTunai menjalankan validasi saldo dan minimum penarikan.

k. Case 4 cek informasi

```
72
73          case 4:
74              if (akunAktif == null) {
75                  System.out.println("Error: Anda belum membuka rekening!");
76              } else {
77                 akunAktif.cekInformasi();
78              }
79              break;
```

Case 4 digunakan untuk menampilkan informasi rekening. Jika belum ada akun aktif, program menampilkan pesan kesalahan. Jika ada, method cekInformasi dipanggil untuk menampilkan nomor rekening, nama pemilik, dan saldo.

l. Case 5 ganti akun

```
80
81          case 5:
82              System.out.print("Masukkan nomor rekening: ");
83              String nomorCari = input.nextLine();
84              boolean ditemukan = false;
```

Case 5 meminta nomor rekening yang akan dicari. Variabel ditemukan diberi nilai false sebagai kondisi awal sebelum pencarian dilakukan. Selanjutnya program memeriksa semua rekening yang tersimpan.

m. Mencari rekening

```
85              for (Rekening rekening : daftarRekening) {
86
87                  if (rekening.nomorRekening.equals(nomorCari)) {
88
89                      akunAktif = rekening;
90                      ditemukan = true;
91                      System.out.println("Akun berhasil diganti.");
92
93                      break;
94                  }
95              }
```

For-each memeriksa object Rekening satu per satu. equals digunakan untuk membandingkan nomor rekening yang tersimpan dengan input pengguna. Jika cocok, akunAktif diganti, ditemukan menjadi true, pesan keberhasilan ditampilkan, lalu break menghentikan pencarian.

n. Menangani rekening tidak ditemukan

```
if (!ditemukan) {
    System.out.println("Rekening tidak ditemukan.");
}
break;
```

Tanda ! membalik nilai boolean. Jika ditemukan masih false, berarti nomor rekening tidak ditemukan dan program menampilkan pesan. Setelah proses case 5 selesai, break mengembalikan program ke menu.

o. Case 6 cetak mutase

```
102            case 6:
103                if (akunAktif == null) {
104                    System.out.println("Error: Anda belum membuka rekening!");
105                } else {
106                    akunAktif.cetakMutasi();
107                }
108                break;
```

Case 6 menghubungkan menu utama dengan fitur cetakMutasi. Program memeriksa akun aktif terlebih dahulu, kemudian memanggil method cetakMutasi jika rekening tersedia. Dengan cara ini pengguna dapat melihat seluruh riwayat transaksi rekening aktif.

p. Case 0 keluar

```
110            case 0:
111                isRunning = false;
112                System.out.println("Sistem ditutup. Terima kasih!");
113                break;
```

Case 0 digunakan untuk menghentikan program. Nilai isRunning diubah

menjadi false sehingga kondisi while tidak lagi terpenuhi. Pesan penutup kemudian ditampilkan sebelum program keluar dari perulangan.

q. Default dan penutupan input

```
            default:
                System.out.println("Pilihan tidak valid!");
            }
        }

        input.close();
    }
}
```

Default dijalankan apabila pilihan pengguna tidak sesuai dengan case yang tersedia.

Setelah perulangan selesai, input.close digunakan untuk menutup Scanner.

Penutupan tersebut menandai bahwa program sudah tidak menggunakan sumber input lagi.

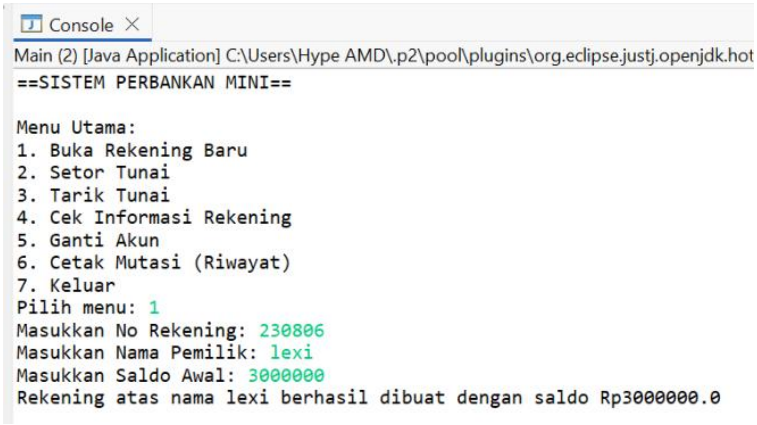
C. HASIL PENGUJIAN

2.9 Hasil Pengujian Praktikum

Bisa. Kalau bagian 2.9 Hasil Pengujian Praktikum mau dibuat dari awal sampai **akhir berdasarkan semua screenshot yang kamu kirim, susunannya seperti ini. Ini sudah** memasukkan pengujian rekening kosong yang terakhir.

2.9 Hasil Pengujian Praktikum

a. Pengujian 1 – Membuka Rekening Baru



Pada pengujian pertama, dipilih menu 1. Buka Rekening Baru. Data yang dimasukkan adalah nomor rekening 230806, nama pemilik lexi, dan saldo awal sebesar Rp3.000.000. Setelah seluruh data dimasukkan, program menampilkan pesan bahwa rekening atas nama lexi berhasil dibuat dengan saldo awal tersebut.

Hasil ini menunjukkan bahwa proses pembuatan objek Rekening berhasil dilakukan dan akun dapat digunakan untuk transaksi berikutnya.

b. Pengujian 2 – Setor Tunai

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Keluar
Pilih menu: 2
Masukkan nominal setor: 1000000
Setor tunai Rp1000000.0 berhasil. Saldo saat ini: Rp4000000.0
```

Setelah rekening berhasil dibuat, pengujian dilanjutkan dengan memilih menu 2.

Setor Tunai. Nominal yang dimasukkan adalah Rp1.000.000. Program menampilkan bahwa setor tunai berhasil dan saldo berubah dari Rp3.000.000 menjadi Rp4.000.000. Transaksi penyetoran tersebut juga dicatat ke dalam riwayatTransaksi sebagai transaksi Kredit. Pencatatan transaksi setor merupakan

bagian dari pengembangan pada Praktikum 2, yaitu membuat objek Transaksi ketika aktivitas penyetoran berhasil.

c. Pengujian 3 – Tarik Tunai Berhasil

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Keluar
Pilih menu: 3
Masukkan nominal tarik: 500000
Tarik tunai Rp500000.0 berhasil. Saldo saat ini: Rp3500000.0
```

Selanjutnya dipilih menu 3. Tarik Tunai dengan nominal penarikan sebesar Rp500.000. Karena saldo rekening mencukupi, transaksi penarikan berhasil dilakukan. Saldo rekening kemudian berubah dari Rp4.000.000 menjadi Rp3.500.000. Transaksi penarikan tersebut dicatat sebagai Debit dengan ID yang menggunakan prefiks TRX-T-. Pengujian ini membuktikan bahwa Tugas 1, yaitu integrasi rekam jejak penarikan, telah diterapkan pada program.

d. Pengujian 4 – Cek Informasi Rekening

```
Main (2) [Java Application] C:\Users\Hype AMD
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Keluar
Pilih menu: 4
--- INFO REKENING ---
No. Rekening : 230806
Nama Pemilik : lexi
Saldo Akhir  : Rp3500000.0
----------------------
```

Setelah melakukan setor dan tarik tunai, dipilih menu 4. Cek Informasi Rekening.

Program menampilkan nomor rekening 230806, nama pemilik lexi, dan saldo akhir sebesar Rp3.500.000. Hasil tersebut menunjukkan bahwa perubahan saldo dari transaksi sebelumnya telah tersimpan dengan benar pada objek rekening.

e. Pengujian 5 – Ganti Akun

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Keluar
Pilih menu: 5
Masukkan nomor rekening: 230806
Akun berhasil diganti.
```

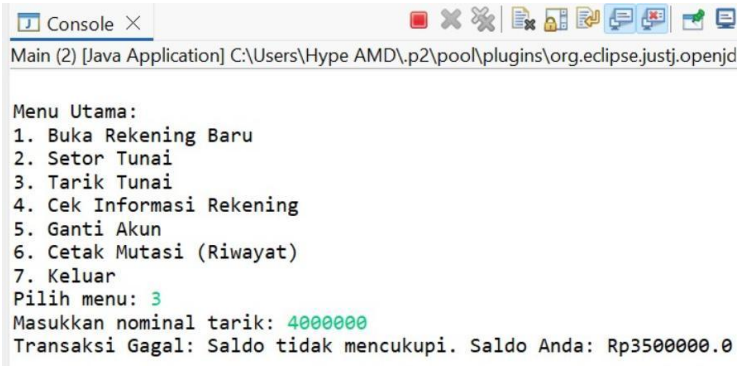
Berikutnya dilakukan pengujian menu 5. Ganti Akun. Nomor rekening 230806 dimasukkan sebagai nomor rekening yang ingin digunakan. Program kemudian menampilkan pesan “Akun berhasil diganti.” Pengujian ini memastikan bahwa program dapat memilih kembali rekening yang tersimpan dalam daftarRekening sebagai akun aktif.

f. Pengujian 6 – Cetak Mutasi Rekening

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Keluar
Pilih menu: 6
==MUTASI REKENING==
ID Transaksi : TRX-S-1 | Jenis : Kredit | Nominal : Rp1000000.0
ID Transaksi : TRX-T-2 | Jenis : Debit | Nominal : Rp500000.0
```

Setelah transaksi setor dan tarik berhasil dilakukan, dipilih menu 6. Cetak Mutasi **(Riwayat). Program menampilkan seluruh transaksi yang tersimpan dalam riwayat** rekening. Hasil yang ditampilkan adalah transaksi TRX-S-1 dengan jenis Kredit sebesar Rp1.000.000 dan transaksi TRX-T-2 dengan jenis Debit sebesar Rp500.000.

g. Pengujian 7 – Tarik Tunai Gagal

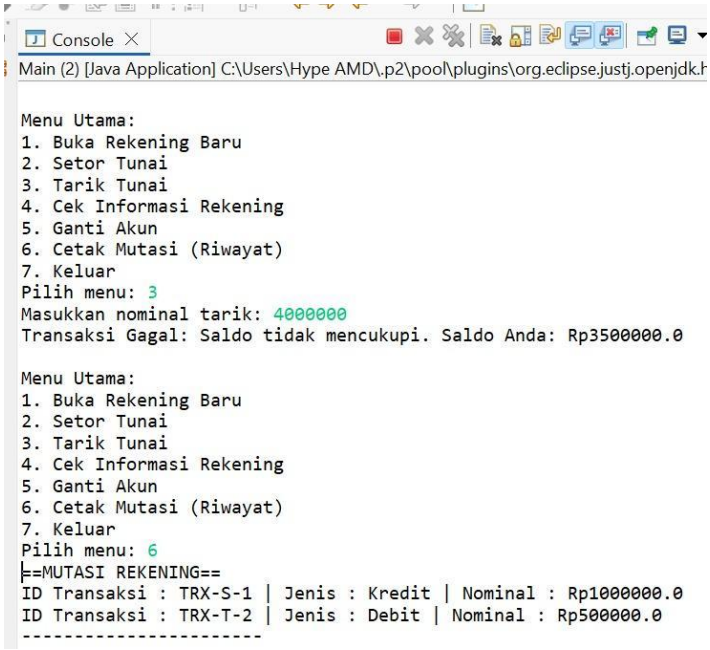


Untuk menguji kondisi penarikan yang tidak berhasil, dilakukan penarikan sebesar Rp4.000.000, sedangkan saldo rekening yang tersedia hanya Rp3.500.000.

Program menolak transaksi tersebut dan menampilkan pesan “Transaksi Gagal:

Saldo tidak mencukupi.” Saldo rekening tetap sebesar Rp3.500.000.Pengujian ini penting karena pada modul dijelaskan bahwa objek transaksi penarikan hanya boleh dicatat apabila penarikan berhasil atau saldo mencukupi.

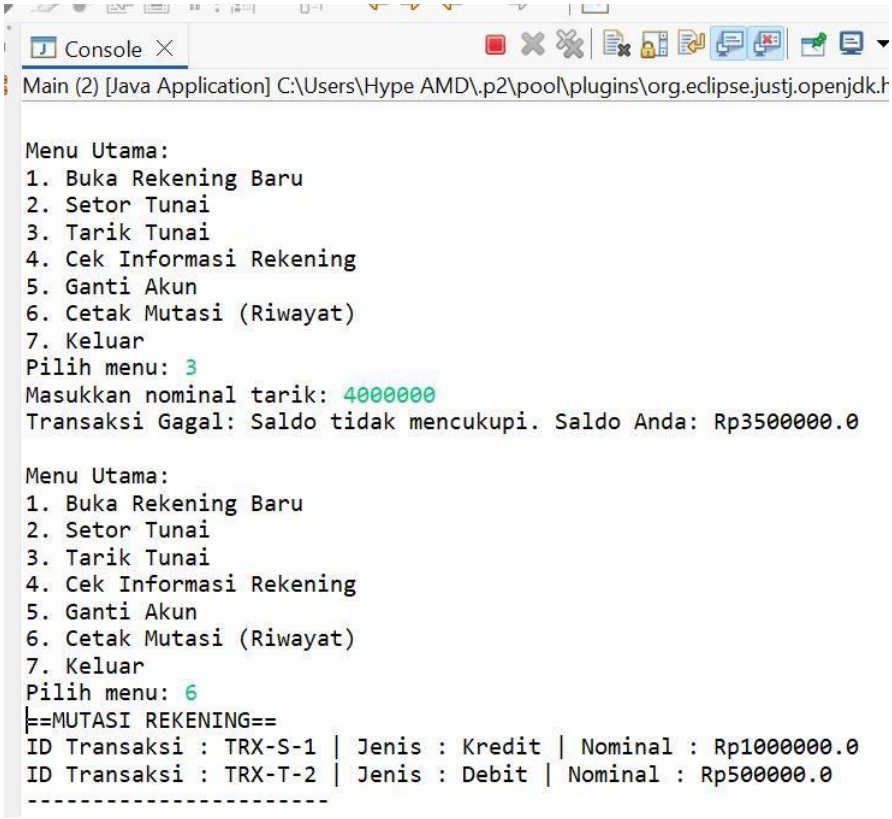
h. Pengujian 8 – Memastikan Penarikan Gagal Tidak Masuk Mutasi



Setelah penarikan sebesar Rp4.000.000 gagal, menu 6. Cetak Mutasi (Riwayat) dipilih kembali. Hasil yang ditampilkan tetap hanya dua transaksi sebelumnya, yaitu TRX-S-1 sebagai Kredit dan TRX-T-2 sebagai Debit. Tidak muncul transaksi baru sebesar Rp4.000.000.

Hasil tersebut menunjukkan bahwa program tidak mencatat penarikan yang gagal ke dalam riwayatTransaksi. Dengan demikian, pencatatan transaksi Debit hanya dilakukan setelah proses penarikan berhasil, sesuai dengan ketentuan Tugas 1 pada modul.

i. Pengujian 9 – Cetak Mutasi pada Rekening Tanpa TransaksiSebagai pengujian terakhir, dibuat rekening baru dengan nomor rekening 12345, nama pemilik nana, dan saldo awal sebesar Rp1.000.000. Setelah rekening berhasil dibuat, tidak dilakukan setor maupun tarik tunai. Pengguna langsung memilih menu 6. Cetak **Mutasi (Riwayat).**



Hasil tersebut menunjukkan bahwa program dapat menangani kondisi ketika riwayatTransaksi masih kosong. Pengujian ini sesuai dengan instruksi Tugas 2 pada modul yang meminta program menampilkan pesan tersebut apabila belum terdapat transaksi pada rekening.

j. Kesimpulan Hasil Pengujian

Berdasarkan seluruh pengujian yang telah dilakukan, program dapat menjalankan proses pembuatan rekening, setor tunai, tarik tunai, pengecekan informasi rekening, pergantian akun, dan pencetakan mutasi. Selain itu, Tugas 1 berhasil diuji dengan menunjukkan bahwa penarikan yang berhasil dicatat sebagai Debit menggunakan prefiks TRX-T-, sedangkan penarikan yang gagal tidak dicatat. Tugas 2 juga berhasil diuji karena program dapat menampilkan seluruh riwayat transaksi dan memberikan pesan “Belum ada transaksi pada rekening ini” ketika riwayat masih kosong. Dengan demikian, hasil pengujian sudah mencakup bagian-bagian utama yang diperintahkan dalam modul Praktikum 2.

PRAKTIKUM 3

RELASI ANTAR OBJEK DAN TRANSAKSI REKENING

A. LATIHAN

2.1 Persiapan Project dan Package

Praktikum 3 menggunakan project PrakPBO_B_2511531006 dan membuat package baru bernama pekan3. Package ini digunakan untuk menyimpan class Transaksi, Rekening, dan Main. Langkahnya dibuat sama seperti susunan pada Laprak 2, tetapi nama package disesuaikan dengan Praktikum 3.

a. Package pada class

```
1 package pekan3;
2
```

Baris package menentukan lokasi class dalam struktur project Java.Package pekan3 memisahkan class Praktikum 3 dari package praktikum sebelumnya.

b. Pembagian fungsi class

Class Transaksi menyimpan data satu transaksi seperti ID, jenis, dan nominal.Class Rekening menyimpan data rekening serta kumpulan riwayat transaksi.Class Main menerima input pengguna dan mengatur menu program.Ketiga class tersebut bekerja bersama membentuk sistem perbankan mini.

2.2 Class Transaksi

Class Transaksi digunakan untuk mewakili satu transaksi yang terjadi pada rekening.

Seluruh bagian class dijelaskan mulai dari package, atribut, constructor, getter, sampai method cetakDetail().

a. Package dan deklarasi class

```
1  package pekan3;
2
3  public class Transaksi {
```

Baris package menempatkan class Transaksi pada package pekan3.public class Transaksi mendeklarasikan class yang dapat digunakan class lain.Kurung kurawal membuka bagian isi class Transaksi.Class ini menjadi model data untuk setiap transaksi yang dibuat.

b. Atribut transaksi

```
5      private String idTransaksi;
6      private String jenis;
7      private double nominal;
```

idTransaksi digunakan sebagai identitas dari setiap transaksi.jenis digunakan untuk membedakan transaksi Kredit dan Debit.nominal menyimpan jumlah uang yang terlibat dalam transaksi.Keyword private menjaga atribut agar tidak diakses langsung dari class lain.

c.

Constructor Transaksi

```
public Transaksi(String idTransaksi, String jenis, double nominal)
    this.idTransaksi = idTransaksi;
    this.jenis = jenis;
    this.nominal = nominal;
}
```

7

Constructor digunakan untuk memberikan nilai awal saat object Transaksi dibuat.Parameter constructor menerima ID, jenis, dan nominal transaksi.Keyword this menunjukkan atribut yang dimiliki oleh object.Nilai parameter kemudian disimpan ke dalam atribut masing-masing.

d. Getter Transaksi

```
public String getIdTransaksi() {return idTransaksi;  }
public String getJenis() {return jenis; }
public double getNominal() {return nominal; }
```

Getter digunakan untuk mengambil sebuah nilai atribut yang bersifat private.getIdTransaksi() mengembalikan ID transaksi yang tersimpan.getJenis() dan getNominal( ) mengembalikan jenis sertano minal transaksi.Method getter memungkinkan class lain membaca data tanpa mengakses atribut langsung.

e.

Method cetakDetail()

```
    public void cetakDetail() {
        System.out.println("ID Transaksi : " + idTransaksi+ " | Jenis : " + jenis+ " | Nominal : Rp" + nominal);
    }
}
```

Method cetakDetail() digunakan untuk menampilkan informasi di sebuah transaksi.System.out.println menggabungkan ID, jenis, dan nominal menggunakan operator +.Informasi transaksi ditampilkan dalam satu baris pada Console.Method ini nantinya dipanggil saat riwayat transaksi dicetak.

2.3 Relasi Class Rekening dan Transaksi

Class Rekening mempunyai hubungan dengan class Transaksi karena satu rekening Dapat mempunyai banyak transaksi.Hubungan tersebut dibuat menggunakan ArrayList<Transaksi>. Bagian berikut menjelaskan seluruh kode Rekening yang berkaitan dengan data rekening dan transaksi.

a. Import dan deklarasi class

```
1  package pekan3;
2
3  import java.util.ArrayList;
4
5  public class Rekening {
```

package pekan3 menempatkan class Rekening pada package yang sama.ArrayList

diimpor

untuk

menyimpan

banyak

object

Transaksi.public

class

Rekening mendeklarasikan class yang mengelola data rekening.Class ini menjadi tempat utama pengelolaan saldo dan riwayat transaksi.

b. Atribut rekening

```
7      private String nomorRekening;
8      private String namaPemilik;
9      private double saldo;
10     private String pin;
```

nomorRekening

menyimpan

nomor

identitas

rekening

pengguna.namaPemilik menyimpan nama pemilik rekening.saldo menyimpan jumlah uang yang dimiliki rekening.pin menyimpan PIN yang digunakan untuk otentikasi.

c.

Riwayat transaksi

```
private ArrayList<Transaksi> riwayatTransaksi =new ArrayList<Transaksi>();
```

riwayatTransaksi merupakan ArrayList yang berisi object Transaksi.Setiap rekening mempunyai tempat sendiri untuk menyimpan riwayat.Object Transaksi dapat ditambahkan setelah transaksi berhasil dilakukan.

d. Constructor Rekening

```
public Rekening(String nomor, String nama, double saldoAwal, String pin) {
    nomorRekening = nomor;
    namaPemilik = nama;
    saldo = saldoAwal;
    this.pin = pin;
    System.out.println("Rekening atas nama " + namaPemilik+ " berhasil dibuat dengan saldo Rp" + saldo);
}
```

Constructor menerima nomor, nama, saldo awal, dan PIN dari pengguna.Data tersebut disimpan ke atribut object Rekening yang baru dibuat.System.out.println menampilkan informasi bahwa rekening berhasil dibuat.Constructor dijalankan ketika class Main membuat object Rekening.

e.Getter Rekening

```
public String getNomorRekening() {return nomorRekening;}
public String getNamaPemilik() {return namaPemilik;}
public double getSaldo() {return saldo;}
```

getNomorRekening() digunakan untuk mengambil nomor rekening.getNamaPemilik() digunakan untuk mengambil nama pemilik rekening.getSaldo() digunakan untuk membaca saldo yang tersimpan.Ketiga getter membantu class Main membaca data private Rekening.

2.4 Pencatatan Setor Tunai

Pada bagian ini, pencatatan transaksi setor tunai dijelaskan berdasarkan perubahan pada class Rekening.Method setorTunai() memeriksa agar nominal setoran lebih dari nol sebelum saldo dan riwayat transaksi diperbarui.Jika setoran valid, saldo

bertambah dan object Transaksi Kredit dimasukkan ke riwayatTransaksi.Dengan demikian, transaksi setor yang berhasil dapat tersimpan dan ditampilkan kembali pada fitur mutasi.

B. TUGAS

2.5 Tugas 1 – Integrasi Rekam Jejak Penarikan

Tugas 1 meminta setiap penarikan yang berhasil mempunyai rekam jejak transaksi.

Penarikan gagal tidak boleh dicatat. Karena itu object Transaksi Debit dibuat setelah pemeriksaan saldo dan batas minimum berhasil dilewati.

10

a. Method tarikTunai dan pemeriksaan saldo

```
public void tarikTunai(double nominal) {
    if (nominal > saldo) {
        System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp"+ saldo);
```

Method tarikTunai menerima nominal uang yang ingin ditarik.Kondisi pertama memeriksa apakah nominal melebihi saldo.Jika saldo tidak mencukupi, transaksi dihentikan dan saldo tetap.Pada kondisi ini object Transaksi tidak dibuat dan tidak masuk riwayat.

b. Pemeriksaan minimum penarikan

```
} else if (nominal < 10000) {
    System.out.println("Transaksi Gagal : Minimal nominal penarikan 10.000");
```

Kondisi else if memeriksa batas minimum penarikan.Nominal kurang dari Rp10.000 dianggap tidak memenuhi aturan program.Saldo tidak dikurangi ketika kondisi tersebut terpenuhi.Transaksi gagal juga tidak ditambahkan ke riwayat.

c.

Penarikan berhasil

```
} else {
    saldo -= nominal;
```

Blok else dijalankan jika saldo mencukupi dan nominal memenuhi batas minimum.Operator -= mengurangi saldo sesuai nominal penarikan.Perubahan saldo dilakukan sebelum transaksi Debit dibuat.Setelah saldo berhasil berubah, proses pencatatan dapat dilanjutkan.

d. Membuat transaksi Debit

```
Transaksi transaksiBaru = new Transaksi("TRX-T-" + (riwayatTransaksi.size() + 1),"Debit",nominal);
```

Object Transaksi dibuat untuk merekam penarikan yang berhasil.ID transaksi menggunakan awalan TRX-T- dan nomor urut riwayat.Jenis transaksi diisi Debit karena saldo rekening berkurang.Nominal yang disimpan sama dengan jumlah uang yang ditarik.

e.Menyimpan transaksi penarikan

```
riwayatTransaksi.add(transaksiBaru);
```

Object transaksiBaru dimasukkan ke ArrayList riwayatTransaksi.Penarikan berhasil kemudian dapat dilihat kembali melalui mutasi.Penarikan yang gagal tidak mencapai baris add tersebut.Hal ini membuat riwayat hanya berisi transaksi yang berhasil.

f.Method cekInformasi()

```
public void cekInformasi() {
    System.out.println("--- INFO REKENING ---");
    System.out.println("No. Rekening : " + nomorRekening);
    System.out.println("Nama Pemilik : " + namaPemilik);
    System.out.println("Saldo Akhir  : Rp" + saldo);
}
```

Method cekInformasi() digunakan untuk menampilkan data rekening aktif.Program mencetak nomor rekening dan nama pemilik.Program juga mencetak saldo akhir yang tersimpan pada object.Method ini dipanggil dari menu Cek Informasi Rekening pada Main.

2.6 Tugas 2 – Fitur Cetak Mutasi Rekening

Tugas 2 menambahkan method cetakMutasi pada class Rekening. Method ini membaca seluruh object Transaksi yang tersimpan dalam ArrayList. Jika belum ada transaksi, program menampilkan pesan khusus.

a. Method cetakMutasi

```
public void cetakMutasi() {
    System.out.println("--- MUTASI REKENING ---");
```

Method
cetakMutasi()
digunakan
untuk
menampilkan
riwayat
transaksi rekening.Baris println pertama memberikan judul pada tampilan mutasi.Riwayat yang ditampilkan berasal dari ArrayList milik rekening aktif.Method ini dijalankan setelah pengguna melewati pemeriksaan PIN.

b. Memeriksa riwayat kosong

```
if (riwayatTransaksi.isEmpty()) {
    System.out.println("Belum ada transaksi pada rekening ini");
```

isEmpty() digunakan untuk memeriksa apakah ArrayList tidak memiliki data.Jika
riwayat
kosong,
program
tidak
menjalankan
perulangan
transaksi.Program menampilkan pesan bahwa belum ada transaksi pada rekening.Kondisi ini penting untuk rekening baru yang belum melakukan transaksi.

c.

Menampilkan seluruh transaksi

```
} else {
    for (Transaksi transaksi : riwayatTransaksi) {
        transaksi.cetakDetail();
    }
}
```

For-each mengambil object Transaksi satu per satu dari riwayat.Variabel transaksi mewakili object yang sedang diproses.Method cetakDetail() dipanggil untuk

menampilkan

detail

transaksi.Semua

transaksi

dalam

ArrayList

akhirnya ditampilkan secara berurutan.

d. Penutup output

```
75
76          System.out.println();
77      }
```

Baris
ini
memberikan
baris
kosong
setelah
daftar
mutasi
selesai ditampilkan.Tujuannya agar tampilan Console tidak terlalu rapat dengan menu berikutnya.Kurung kurawal menutup method cetakMutasi setelah seluruh proses selesai.Method kemudian kembali ke bagian program yang memanggilnya.

2.7 Penambahan Menu 6 pada Main

Agar fitur cetak mutasi dapat digunakan, menu utama pada class Main mempunyai pilihan nomor 6. Ketika dipilih, program memeriksa akun aktif, meminta PIN, dan memanggil cetakMutasi jika PIN benar.

a. Menambahkan menu

```
System.out.println("6. Cetak Mutasi (Riwayat)");
System.out.println("7. Keluar");
```

Baris tersebut menampilkan pilihan Cetak Mutasi pada menu utama.Nomor 6 digunakan untuk membuka riwayat transaksi rekening aktif.Nomor 7 digunakan untuk menghentikan program.Kedua pilihan tersebut dijalankan melalui switch pada Main.

b. Case 6

```
case 6:
    if (akunAktif == null) {
        System.out.println("Error: Anda belum membuka rekening!");
```

case 6 dijalankan ketika pengguna memilih menu Cetak Mutasi.Program terlebih dahulu memeriksa apakah akunAktif masih null.Jika belum ada rekening, program menampilkan pesan kesalahan.Jika ada rekening, proses dilanjutkan ke pemeriksaan

PIN.

c.

Memanggil cetakMutasi

```
} else {
    System.out.print("Masukkan PIN: ");
    String pinMutasi = input.nextLine();

    if (akunAktif.otentikasi(pinMutasi)) {
        akunAktif.cetakMutasi();
```

Program meminta PIN sebelum riwayat transaksi ditampilkan.Method otentikasi() membandingkan PIN input dengan PIN rekening.Jika PIN benar, method cetakMutasi() dipanggil pada akun aktif.Jika PIN salah, program menolak akses dan tidak mencetak mutasi.

2.8 Integrasi Program pada Class Main

Setelah fitur tugas selesai dibuat pada class Rekening, perubahan tersebut dihubungkan ke menu pada class Main. Bagian ini menjelaskan kode Main secara bertahap pada bagian yang berkaitan dengan input PIN, autentikasi, dan menu fitur baru.

a. Input PIN saat membuka rekening

```
System.out.print("Masukkan PIN (6 digit): ");
String pin = input.nextLine();
```

Program meminta pengguna memasukkan PIN ketika memilih menu pembukaan rekening.PIN disimpan ke dalam variabel pin sebelum object Rekening dibuat.Nilai tersebut kemudian diteruskan ke constructor Rekening bersama data rekening lainnya.Dengan cara ini setiap rekening yang dibuat memiliki PIN untuk autentikasi.

b. Case 3 – Tarik tunai dengan autentikasi

```
String pinTarik = input.nextLine();
if (akunAktif.otentikasi(pinTarik)) {
```

Pada case tarik tunai, program meminta PIN dari pengguna sebelum proses penarikan.PIN tersebut diperiksa menggunakan method otentikasi() milik akunAktif.Jika hasilnya true, pengguna dapat melanjutkan dengan memasukkan nominal penarikan.Jika PIN salah, program menampilkan pesan akses ditolak dan transaksi tidak diproses.

c.case 6 – Cetak mutasi dengan autentikasi

```
String pinMutasi = input.nextLine();
if (akunAktif.otentikasi(pinMutasi)) {
```

Case 6 meminta PIN sebelum riwayat transaksi ditampilkan.Program menggunakan otentikasi() untuk memastikan PIN sesuai dengan rekening aktif.Jika autentikasi berhasil, method cetakMutasi() dipanggil untuk menampilkan riwayat.Jika PIN salah, program tidak melanjutkan proses pencetakan mutasi.

d. Menu 6 dan menu keluar

```
System.out.println("6. Cetak Mutasi (Riwayat)");
System.out.println("7. Keluar");
```

Menu 6 ditambahkan agar pengguna dapat mengakses fitur cetak mutasi.Menu 7 digunakan untuk mengakhiri perulangan menu program.Saat pilihan 7 dijalankan, nilai isRunning diubah menjadi false.Perubahan tersebut membuat program keluar dari perulangan utama.

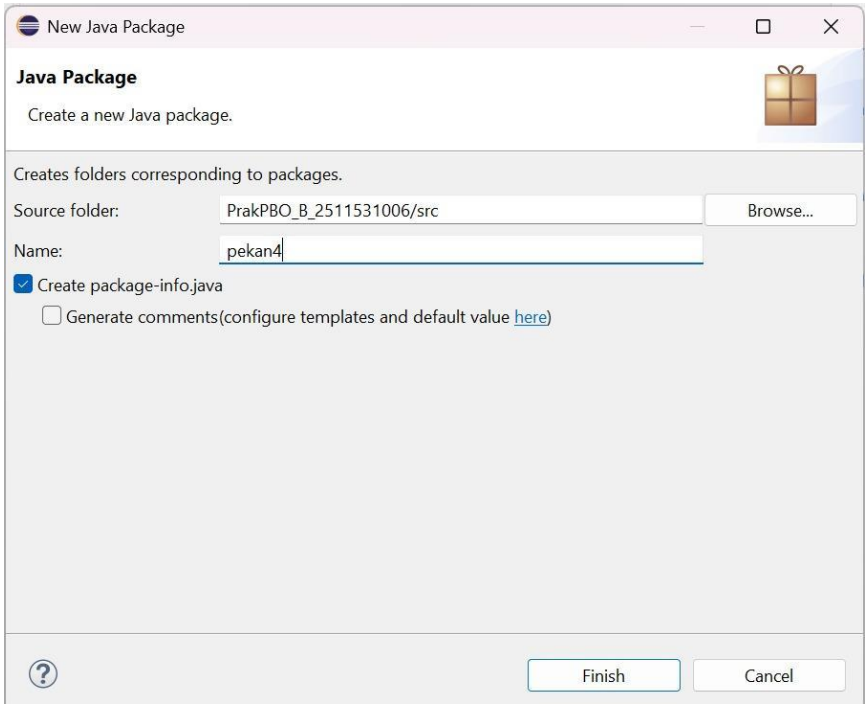
PRAKTIKUM 4

EKSPANSI PRODUK KEUANGAN (INHERITANCE)

A. LATIHAN

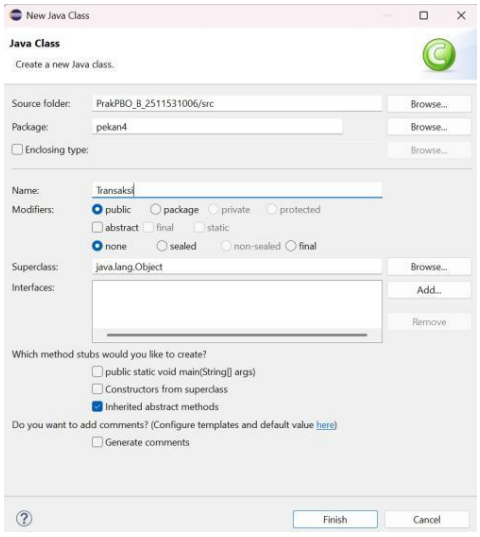
2.1 Persiapan Project dan Package

Project yang digunakan adalah project praktikum sebelumnya, kemudian dibuat package baru bernama pekan4. Package ini digunakan untuk menempatkan class pada Praktikum 4 agar tidak bercampur dengan pekerjaan praktikum sebelumnya. Class yang digunakan terdiri dari Rekening, Transaksi, RekeningTabungan, RekeningGiro, dan Main. Rekening menjadi superclass, sedangkan dua class produk menjadi subclass.



Gambar 2.1 Pembuatan package pekan4

Setelah package dibuat, class Rekening disiapkan sebagai class induk dan dua class baru dibuat untuk produk rekening. Class Transaksi tetap digunakan untuk menyimpan detail transaksi, sedangkan Main digunakan untuk menjalankan program. Susunan tersebut sesuai dengan pengembangan program pada modul Praktikum 4.



Gambar 2.2 Pembuatan class Transaksi pada package pekan4

2.2 Refactoring Class Rekening sebagai Superclass

Langkah pertama pada modul adalah melakukan refactoring terhadap class Rekening.

Rekening dijadikan superclass karena data seperti nomor rekening, nama pemilik, saldo, PIN, dan riwayat transaksi merupakan bagian yang sama pada produk Tabungan maupun Giro.

Dengan menjadikannya superclass, subclass dapat menggunakan kembali method dasar rekening. Perubahan ini juga mengurangi pengulangan kode.

a. Package, import, dan deklarasi class

```
1  package pekan4;
2
3  import java.util.ArrayList;
4
5  public class Rekening {
```

Baris package menunjukkan bahwa class Rekening berada pada package pekan4.

Import ArrayList digunakan karena class Rekening menyimpan riwayat transaksi 5

dalam bentuk kumpulan object Transaksi.Deklarasi public class Rekening menunjukkan bahwa class dapat digunakan oleh class lain dalam program.Pada Praktikum 4 class ini berperan sebagai superclass yang akan diwarisi oleh RekeningTabungan dan RekeningGiro.

b. Atribut class Rekening

```
7      private String nomorRekening;
8      private String namaPemilik;
9      protected double saldo;
10     private String pin;
```

Atribut nomorRekening, namaPemilik, dan pin tetap menggunakan private karena merupakan data internal rekening. Atribut saldo diubah menjadi protected agar dapat

digunakan oleh subclass yang membutuhkan akses terhadap saldo.Atribut riwayatTransaksi juga menggunakan protected sehingga subclass tetap dapat mengakses riwayat yang diwariskan. Penggunaan protected merupakan bagian penting dari penerapan inheritance pada modul.

c. Constructor Rekening

```
11
12      protected ArrayList<Transaksi> riwayatTransaksi = new ArrayList<Transaksi>();
13
14      public Rekening(String nomor, String nama, double saldoAwal, String pin) {
15          nomorRekening = nomor;
16          namaPemilik = nama;
17          saldo = saldoAwal;
18          this.pin = pin;
19          System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat dengan saldo Rp" + saldo);
20      }
21
22      public String getNomorRekening() {return nomorRekening;}
23      public String getNamaPemilik() {return namaPemilik;}
24      public double getSaldo() {return saldo;}
```

Constructor Rekening digunakan untuk mengisi bagian data yang bersifat umum pada setiap rekening. Nilai nomor, nama, saldo awal, dan PIN diterima melalui parameter constructor. Nilai tersebut kemudian disimpan ke atribut object yang dibuat.

Constructor ini nantinya dipanggil oleh subclass menggunakan keyword super().

d. Method setorTunai()

```
26      public void setorTunai(double nominal) {
27          if (nominal > 0) {
28              saldo += nominal;
29
30              Transaksi transaksiBaru = new Transaksi("TRX-S-" + (riwayatTransaksi.size() + 1),"Kredit",nominal);
31              riwayatTransaksi.add(transaksiBaru);
32
33              System.out.println("Setor tunai Rp" + nominal +" berhasil. Saldo saat ini: Rp" + saldo);
34          } else {
35              System.out.println("Gagal: Nominal setor harus lebih dari 0!");
36          }
37      }
```

Pada method setorTunai(),saldo akan bertambah sesuai nominal yang dimasukkan apabila nominal memenuhi validasi. Setelah saldo berubah, program membuat object Transaksi baru dengan jenis Kredit. Object tersebut dimasukkan ke dalam

riwayatTransaksi sehingga transaksi yang berhasil tetap dapat dilihat kembali.Bagian ini tetap berada di superclass karena setor tunai merupakan perilaku umum semua rekening.

e. Method tarikTunai()

```
public void tarikTunai(double nominal) {
    if (nominal > saldo) {
        System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp" + saldo);
    } else if (nominal < 10000) {
        System.out.println("Transaksi Gagal : Minimal nominal penarikan 10.000");
    } else {
        saldo -= nominal;
        Transaksi transaksiBaru = new Transaksi("TRX-T-" + (riwayatTransaksi.size() + 1),"Debit",nominal );
        riwayatTransaksi.add(transaksiBaru);
        System.out.println("Tarik tunai Rp" + nominal +" berhasil. Saldo saat ini: Rp" + saldo);
    }
}
```

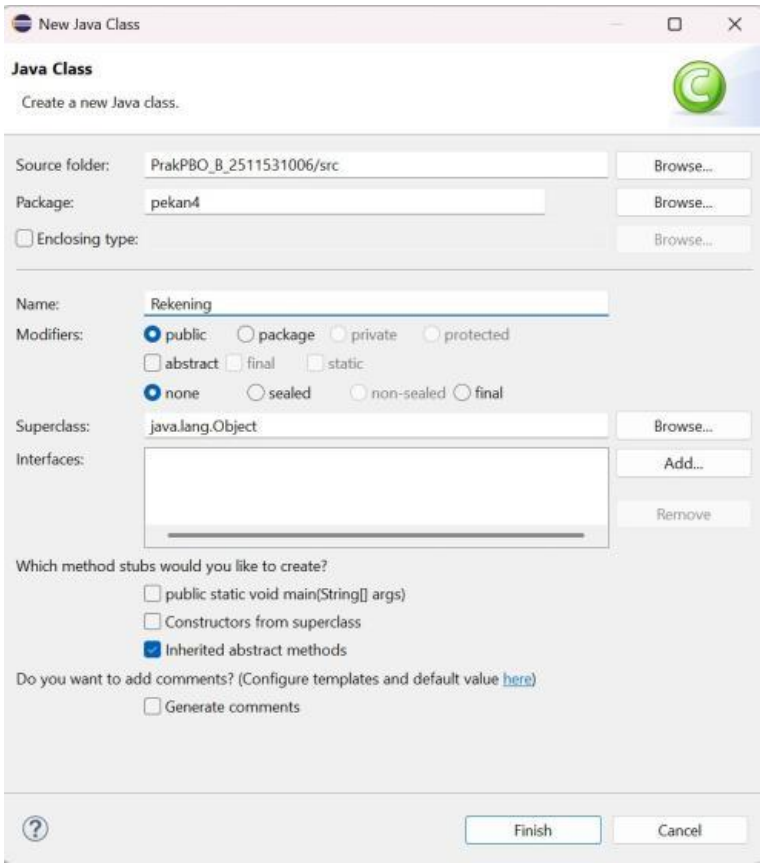
Method tarikTunai() digunakan untuk mengurangi saldo dan mencatat transaksi penarikan. Sebelum bagian pengurangan saldo dijalankan, program melakukan validasi agar nominal tidak melebihi saldo dan tidak kurang dari Rp10.000. Jika transaksi berhasil, jenis transaksi dicatat sebagai Debit. Method ini diwariskan oleh subclass sehingga Tabungan dan Giro tetap memiliki fungsi transaksi dasar.

f. Informasi, mutasi, dan autentikasi

```
public void cekInformasi() {
    System.out.println("--- INFO REKENING ---");
    System.out.println("No. Rekening : " + nomorRekening);
    System.out.println("Nama Pemilik : " + namaPemilik);
    System.out.println("Saldo Akhir  : Rp" + saldo);
}

public void cetakMutasi() {
    System.out.println("--- MUTASI REKENING ---");
    if (riwayatTransaksi.isEmpty()) {
        System.out.println("Belum ada transaksi pada rekening ini");
    } else {
        for (Transaksi transaksi : riwayatTransaksi) {
            transaksi.cetakDetail();
        }
    }
    System.out.println();
}
```

Method cekInformasi() digunakan untuk menampilkan nomor rekening, nama pemilik, dan saldo. Method cetakMutasi() membaca riwayatTransaksi dan menampilkan detail transaksi yang tersimpan. Jika belum terdapat transaksi, program memberikan pesan bahwa belum ada transaksi pada rekening tersebut. Method otentikasi() membandingkan PIN yang dimasukkan dengan PIN rekening sebelum fitur tertentu dijalankan.



Gambar 2.3 Pembuatan class Rekening pada package pekan4

2.3 Class RekeningTabungan

Class RekeningTabungan dibuat sebagai subclass dari Rekening sesuai langkah pada modul.

Produk ini memiliki seluruh kemampuan dasar Rekening, tetapi ditambahkan suku bunga dan simulasi bunga akhir bulan. Dengan inheritance, kode setor, tarik, informasi, dan mutasi tidak perlu dibuat ulang. Class hanya berisi bagian yang menjadi ciri khusus tabungan.

a. Deklarasi subclass

```
1  package pekan4;
2
3  public class RekeningTabungan extends Rekening {
```

Keyword extends menunjukkan hubungan pewarisan antara RekeningTabungan dan Rekening.Artinya RekeningTabungan merupakan jenis Rekening sehingga dapat menggunakan method yang berasal dari superclass. Hubungan ini sesuai konsep Is- A yang dijelaskan dalam modul. Bagian khusus tabungan kemudian ditambahkan di dalam class tersebut.

b. Atribut sukuBunga

```
4
5      private double sukuBunga;
```

Atribut sukuBunga digunakan untuk menyimpan persentase bunga rekening tabungan. Atribut ini bersifat private karena hanya menjadi data khusus milik RekeningTabungan.Nilainya diperoleh saat pengguna memilih produk Tabungan pada menu pembukaan rekening. Data tersebut kemudian digunakan oleh method simulasi bunga.

c. Constructor dan super()

```
    public RekeningTabungan(
        String nomor,
        String nama,
        double saldoAwal,
        String pin,
        double sukuBunga) {

         super(nomor, nama, saldoAwal, pin);
         this.sukuBunga = sukuBunga;
    }

    public double getSukuBunga() {return sukuBunga;}
```

9

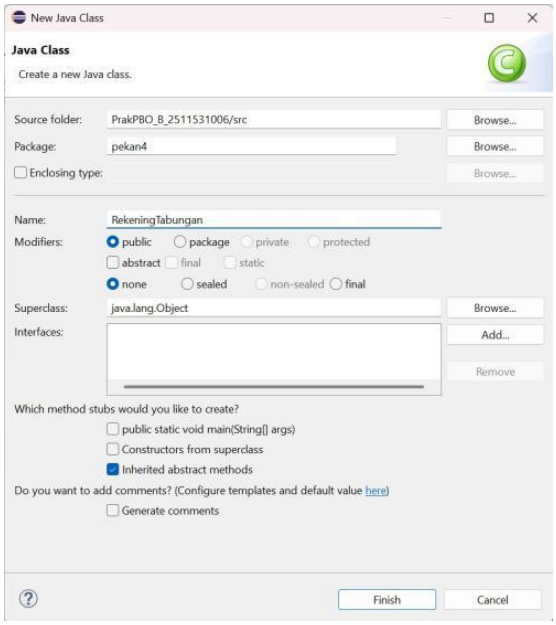
Pemanggilan super() digunakan untuk menjalankan constructor class Rekening. Baris ini harus berada pada bagian awal constructor subclass agar data umum rekening dapat diinisialisasi terlebih dahulu. Setelah constructor induk dijalankan, nilai sukuBunga disimpan sebagai data khusus tabungan. Dengan demikian object memiliki data umum sekaligus data khusus produk.

d. Method tambahBungaAkhirBulan()

```
    public void tambahBungaAkhirBulan() {
        double bunga = saldo * sukuBunga / 100;
        saldo += bunga;
        System.out.println("Bunga akhir bulan sebesar Rp" + bunga +" telah ditambahkan.");
        System.out.println("Saldo setelah bunga: Rp" + saldo);
    }
}
```

Method ini menghitung bunga berdasarkan saldo yang dimiliki rekening dan persentase suku bunga.Hasil perhitungan disimpan pada variabel nominalBunga, kemudian ditambahkan ke saldo. Setelah itu program membuat object Transaksi berjenis Bunga dengan ID yang diawali TRX-B dan memasukkannya ke riwayatTransaksi, sehingga bunga juga muncul pada mutasi rekening sesuai contoh pada modul. Method tersebut hanya berada pada RekeningTabungan karena Giro

tidak mempunyai fitur bunga bulanan dalam perintah modul. Fitur ini nantinya dipanggil melalui menu 7 pada class Main.



Gambar 2.4 Pembuatan class RekeningTabungan pada package pekan4

2.4 Class RekeningGiro

RekeningGiro dibuat sebagai subclass kedua dari Rekening. Berbeda dengan Tabungan, class ini menambahkan data batas overdraft sebagai karakteristik produk Giro. Bagian umum seperti saldo, PIN, setor, tarik, dan mutasi tetap diperoleh dari superclass. Dengan begitu class Giro tidak perlu mengulang kode dasar yang sudah ada.

a. Deklarasi dan atribut

```
1  package pekan4;
2
3  public class RekeningGiro extends Rekening {
4
5      private double batasOverdraft;
```

Deklarasi extends membuat RekeningGiro mewarisi class Rekening.Atribut batasOverdraft menyimpan batas overdraft yang dimasukkan ketika pengguna

membuat Giro.Atribut ini dibuat private karena merupakan data khusus Giro. Pada tahap praktikum ini, modul meminta data tersebut disimpan sebagai karakteristik produk.

b. Constructor RekeningGiro

```
public RekeningGiro(String nomor,String nama,double saldoAwal,String pin,double batasOverdraft) {
    super(nomor, nama, saldoAwal, pin);
    this.batasOverdraft = batasOverdraft;
}
```

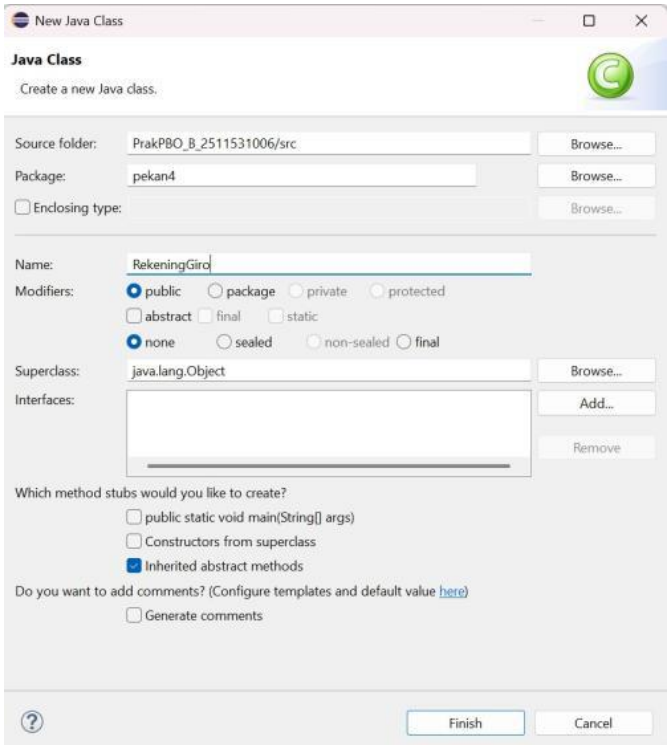
Constructor Giro memanggil constructor superclass menggunakan super() untuk mengisi data umum rekening. etelah itu nilai batas overdraft dimasukkan ke atribut khusus Giro.Cara ini menunjukkan bahwa subclass tidak membuat ulang proses inisialisasi nomor, nama, saldo, dan PIN. Bagian umum tetap ditangani oleh superclass.

c. Getter batasOverdraft

```
12      public double getBatasOverdraft() {
13          return batasOverdraft;
14      }
15  }
```

Getter digunakan untuk mengambil nilai batas overdraft dari object RekeningGiro. Method ini memungkinkan bagian program lain membaca data tersebut tanpa 11

mengubah atribut secara langsung.Penggunaan getter juga menjaga atribut tetap private.Pada Praktikum 4, batas overdraft terutama digunakan sebagai data tambahan ketika produk Giro dibuat.



Gambar 2.5 Pembuatan class RekeningGiro pada package pekan4

2.5 Penerapan Upcasting

Setelah dua subclass dibuat, object dari kedua produk dapat disimpan dalam satu ArrayList bertipe Rekening. Inilah bagian yang dimaksud upcasting pada modul. Referensi bertipe superclass dapat menunjuk object dari subclass karena subclass merupakan turunan dari superclass. Dengan cara tersebut program tidak membutuhkan ArrayList terpisah untuk Tabungan dan Giro.

a. Koleksi rekening

```
ArrayList<Rekening> daftarRekening = new ArrayList<Rekening>();
```

ArrayList tersebut menggunakan tipe Rekening sebagai tipe elemennya. Karena Rekening merupakan superclass, object RekeningTabungan dan RekeningGiro dapat dimasukkan ke dalam daftar. Struktur ini membuat pengelolaan beberapa produk rekening menjadi lebih sederhana. Daftar yang sama juga dapat digunakan ketika pengguna mengganti akun melalui menu Main.

b. Contoh upcasting

```
Rekening rekeningBaru;

rekeningBaru = new RekeningTabungan(no,nama,saldo,pin,sukuBunga);
```

Pada contoh tersebut object yang dibuat sebenarnya adalah RekeningTabungan, tetapi disimpan pada referensi bertipe Rekening. Perubahan referensi tersebut merupakan upcasting dan dapat dilakukan secara otomatis. Program tetap dapat menggunakan method yang diwariskan dari Rekening. Jika fitur khusus Tabungan dibutuhkan, referensi tersebut perlu diperiksa dan dilakukan downcasting.

B. TUGAS

2.6 Tugas 1 – Multi-Produk Rekening

Tugas pertama pada modul meminta perubahan proses pembukaan rekening sehingga pengguna dapat memilih dua produk. Pilihan pertama adalah Tabungan Umum dan pilihan kedua adalah Giro Bisnis. Jika Tabungan dipilih, program meminta sukuBunga, sedangkan jika Giro dipilih program meminta batasOverdraft. Object yang dibuat kemudian disimpan dalam daftarRekening melalui referensi bertipe Rekening.

a. Pilihan produk pada menu 1

```
System.out.println("\nPilih Produk:");
System.out.println("1. Tabungan Umum");
System.out.println("2. Giro Bisnis");
System.out.print("Pilihan produk: ");

int produk = input.nextInt();
input.nextLine();
```

Bagian ini menampilkan dua pilihan produk sesuai instruksi Tugas 1 pada modul.

Input pengguna disimpan dalam produk untuk menentukan subclass yang akan dibuat.Program kemudian menggunakan percabangan untuk memisahkan proses Tabungan dan Giro. Dengan cara ini satu menu pembukaan rekening dapat digunakan untuk dua jenis produk.

b. Membuat object Tabungan

```
if (produk == 1) {

    System.out.print("Masukkan Suku Bunga (%): ");
    double sukuBunga = input.nextDouble();
    input.nextLine();

    rekeningBaru = new RekeningTabungan(no,nama,saldo,pin,sukuBunga);
```

Jika pengguna memilih Tabungan, program meminta nilai sukuBunga. Nilai tersebut diberikan kepada constructor RekeningTabungan bersama data rekening lainnya.

Object yang terbentuk kemudian disimpan pada referensi Rekening sehingga terjadi upcasting. Setelah itu object dapat dimasukkan ke daftarRekening dan digunakan sebagai akun aktif.

c. Membuat object Giro

```
} else if (produk == 2) {

    System.out.print("Masukkan Batas Overdraft: ");
    double batasOverdraft = input.nextDouble();
    input.nextLine();

    rekeningBaru = new RekeningGiro(no,nama,saldo,pin,batasOverdraft);

} else {

    System.out.println("Pilihan produk tidak valid!");
    break;
}

daftarRekening.add(rekeningBaru);
akunAktif = rekeningBaru;
```

Jika pengguna memilih Giro, program meminta batas overdraft terlebih dahulu. Nilai tersebut diberikan ke constructor RekeningGiro sebagai atribut khusus produk.

Object Giro tetap disimpan melalui referensi Rekening sehingga dapat dikelola bersama object Tabungan. Hal ini merupakan penerapan utama upcasting pada Tugas 1.

2.7 Tugas 2 – Simulasi Akhir Bulan

Tugas kedua pada modul meminta penambahan menu 7 dengan nama Simulasi Akhir Bulan (Khusus Tabungan). Fitur ini hanya boleh dijalankan ketika akun aktif merupakan RekeningTabungan. Oleh karena itu program perlu memeriksa tipe object sebelum 14

melakukan downcasting. Jika akun aktif merupakan Giro, program harus menampilkan pesan bahwa fitur bunga hanya berlaku untuk Rekening Tabungan.

a. Pemeriksaan instanceof

```
case 7:

    if (akunAktif == null) {
        System.out.println("Error: Anda belum membuka rekening!");

    } else if (akunAktif instanceof RekeningTabungan) {
```

Operator instanceof digunakan untuk memeriksa apakah object yang sedang ditunjuk akunAktif benar-benar merupakan RekeningTabungan. Pemeriksaan ini penting karena akunAktif menggunakan tipe referensi Rekening. Jika hasil pemeriksaan true, program dapat melanjutkan ke proses downcasting. Jika false, program tidak boleh menjalankan method khusus tabungan.

b. Downcasting

```
RekeningTabungan tabungan =(RekeningTabungan) akunAktif;
```

Baris tersebut mengubah referensi bertipe Rekening menjadi referensi RekeningTabungan.Proses ini disebut downcasting karena referensi dikembalikan ke tipe subclass yang lebih spesifik. Downcasting dilakukan setelah instanceof agar tidak terjadi kesalahan ketika object sebenarnya adalah Giro. Setelah berhasil, method khusus Tabungan dapat dipanggil melalui variabel tabungan.

c. Menjalankan simulasi bunga

```
tabungan.tambahBungaAkhirBulan();
```

Setelah downcasting berhasil, method tambahBungaAkhirBulan() dapat dipanggil.

Method menghitung bunga berdasarkan saldo dan sukuBunga yang dimiliki rekening.

Hasil bunga kemudian ditambahkan ke saldo tabungan. Fitur tersebut tidak dipanggil pada Giro karena Giro tidak memiliki method khusus tersebut.

d. Kondisi ketika bukan Tabungan

```
} else {
    System.out.println("Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan.");
}
break;
```

Bagian else dijalankan ketika akun aktif bukan RekeningTabungan. Kondisi ini dapat terjadi ketika pengguna sedang menggunakan RekeningGiro. Program memberikan pesan agar pengguna mengetahui bahwa simulasi bunga hanya tersedia untuk Tabungan. Dengan demikian downcasting hanya dilakukan pada object yang sesuai.

2.8 Integrasi Program pada Class Main

Class Main menjadi tempat seluruh fitur praktikum dihubungkan. Pada Praktikum 4 perubahan utama terdapat pada menu 1 untuk membuat multi-produk dan menu 7 untuk menjalankan simulasi bunga. Menu lama seperti setor tunai, tarik tunai, cek informasi, ganti akun, dan cetak mutasi tetap digunakan. Struktur tersebut membuat hasil Praktikum 4 tetap terhubung dengan program praktikum sebelumnya. Kode lengkap class Main dan seluruh class lainnya dapat dilihat pada bagian Lampiran.

a. Daftar rekening dan akun aktif

```
ArrayList<Rekening> daftarRekening = new ArrayList<Rekening>();
Rekening akunAktif = null;

boolean isRunning = true;
```

daftarRekening digunakan untuk menyimpan seluruh object rekening yang dibuat pengguna. Tipe elemennya Rekening karena object Tabungan dan Giro sama-sama merupakan turunan Rekening.akunAktif digunakan untuk menyimpan rekening yang sedang dipakai. Saat pengguna memilih akun lain melalui menu ganti akun, nilai akunAktif akan berubah.

b. Integrasi menu 1

```
System.out.println("\nMenu Utama:");
System.out.println("1. Buka Rekening Baru");
```

16

Menu 1 menggabungkan data umum rekening dengan pilihan produk.Setelah nomor rekening,nama,saldo, dan PIN dimasukkan, program meminta pengguna memilih Tabungan atau Giro.Data tambahan produk kemudian dimasukkan sesuai pilihan.

Object yang dibuat dimasukkan ke daftarRekening sehingga konsep upcasting benar-benar digunakan pada program.

c. Integrasi menu 2 sampai menu 6

```
System.out.println("2. Setor Tunai");
System.out.println("3. Tarik Tunai");
System.out.println("4. Cek Informasi Rekening");
System.out.println("5. Ganti Akun");
System.out.println("6. Cetak Mutasi (Riwayat)");
```

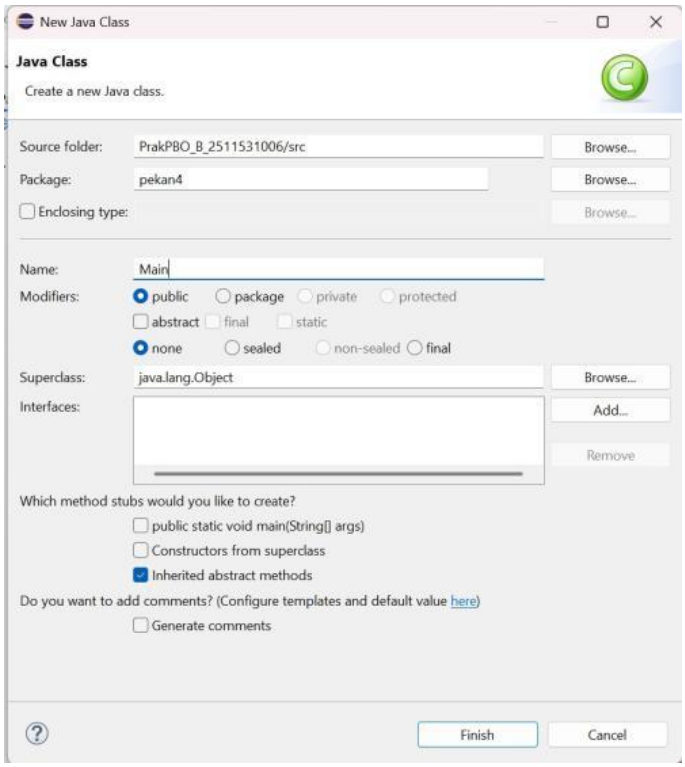
Menu 2 tetap memanggil setorTunai(), sedangkan menu 3 melakukan autentikasi sebelum memanggil tarikTunai(). Menu 4 menampilkan informasi rekening aktif dan menu 5 mencari rekening berdasarkan nomor rekening. Menu 6 melakukan autentikasi lalu memanggil cetakMutasi(). Method tersebut berasal dari superclass sehingga dapat digunakan oleh kedua produk rekening.

d. Integrasi menu 7 dan menu 8

```
System.out.println("7. Simulasi Akhir Bulan (Khusus Tabungan)");
System.out.println("8. Keluar");
```

Menu 7 yaitu menjalankan sebuah pemeriksaan instanceof,downcasting,dan tambahBungaAkhirBulan() sesuai Tugas 2 pada modul. Menu 8 digunakan untuk menghentikan program dengan mengubah nilai isRunning menjadi false. Jika pengguna memasukkan pilihan di luar menu yang tersedia, bagian default

menampilkan pesan pilihan tidak valid. Dengan demikian seluruh perubahan Praktikum 4 tetap terintegrasi dalam Main.

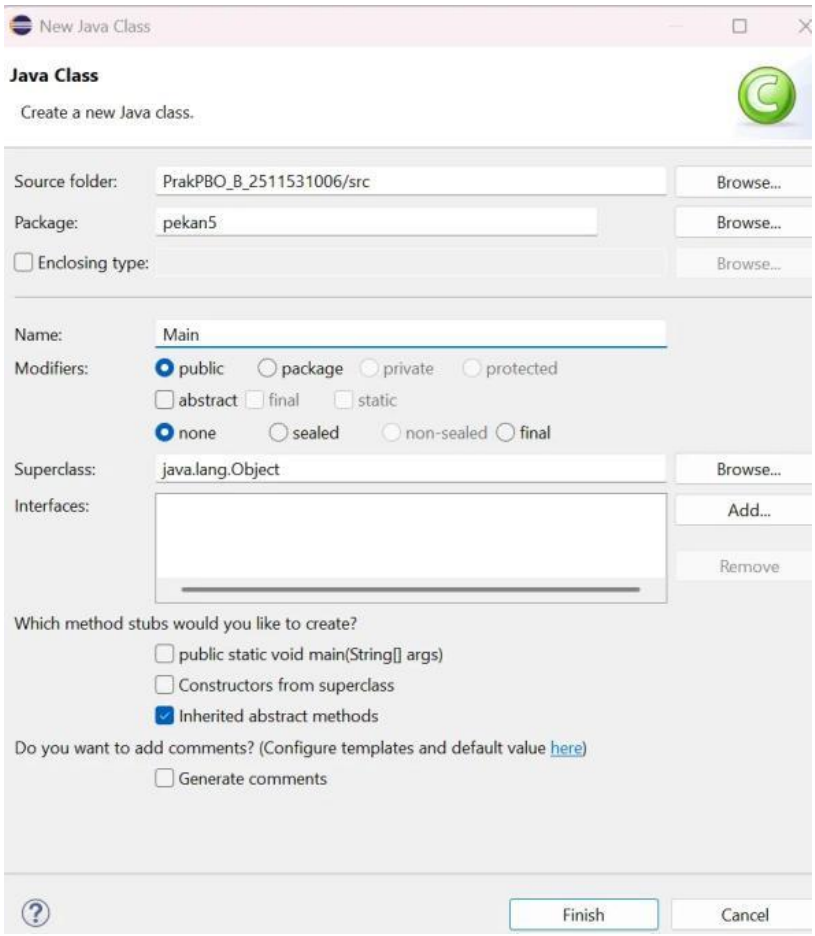


Gambar 2.6 Pembuatan class Main pada package pekan4

PRAKTIKUM 5

TRANSAKSI DINAMIS (POLYMORPHISM)

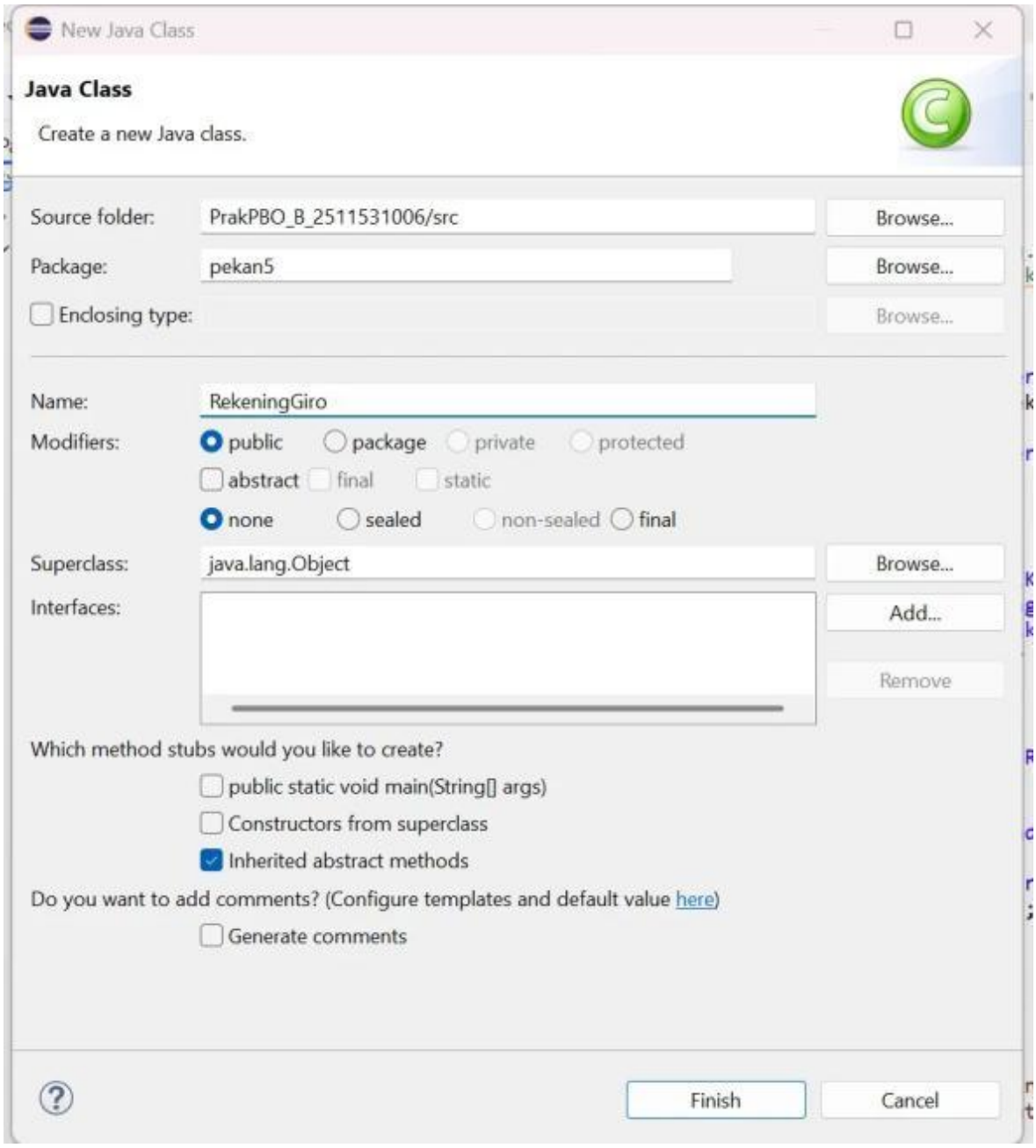
Class Main dibuat sebagai pusat menu dan input pengguna. Opsi public static void main tidak dicentang karena method main ditulis sendiri pada kode.



Gambar 2.5 Pembuatan class Main

g. Pembuatan class RekeningGiro

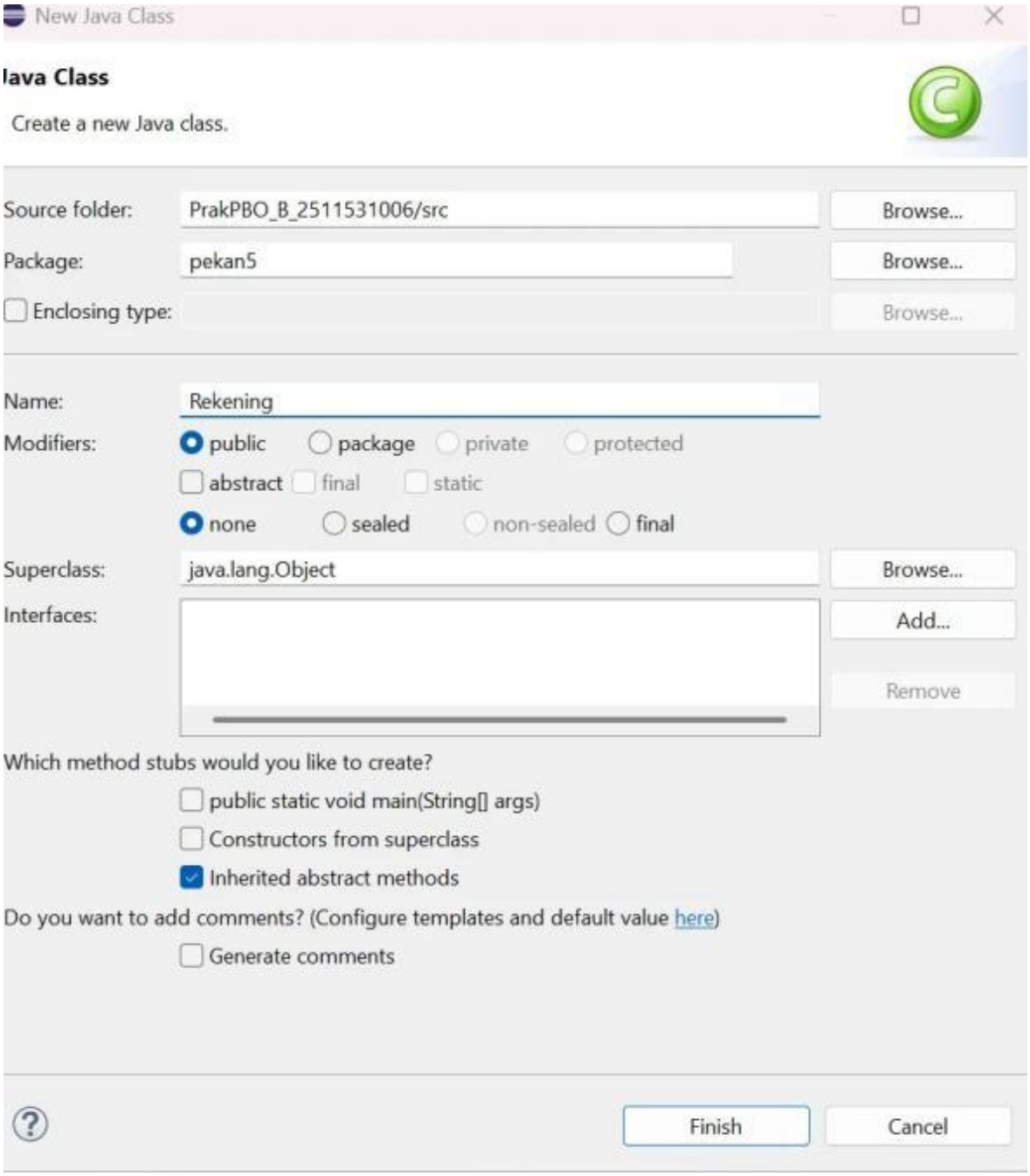
Class RekeningGiro dibuat pada package pekan5. Class ini juga mewarisi Rekening dan menimpa method tarikTunai.



Gambar 2.6 Pembuatan class RekeningGiro

h. Pembuatan class Rekening

Class Rekening dibuat sebagai class induk yang mengelola saldo, riwayat transaksi, setor tunai, tarik tunai, dan transfer.



Gambar 2.7 Pembuatan class Rekening

2.2 Langkah 1 – Refaktor Method tarikTunai pada Kelas Induk (Rekening.java)

Langkah 1 mengubah method tarikTunai pada class Rekening. Tipe kembalian diubah dari void menjadi boolean agar method ini dapat dipakai oleh fitur transfer.

Method mengembalikan true jika penarikan berhasil dan false jika gagal. Aturan minimal penarikan Rp10.000 dari praktikum sebelumnya tetap dipertahankan.

a. Atribut protected pada class Rekening

```
7       private String nomorRekening;
8       private String namaPemilik;
9       protected double saldo;
10      private String pin;
```

Atribut nomorRekening, namaPemilik, dan pin tetap private agar tidak bisa diakses langsung dari class lain. Atribut saldo diubah menjadi protected supaya dapat dipakai oleh class anak. Hal ini dibutuhkan karena RekeningGiro harus membaca dan mengubah saldo pada method tarikTunai miliknya. ArrayList 8

riwayatTransaksi juga protected agar RekeningGiro dapat mencatat transaksi penarikan. Atribut protected bisa diakses oleh class anak, sedangkan atribut private hanya bisa diakses oleh class itu sendiri.

b. Deklarasi method tarikTunai

```
50
51⊖     public boolean tarikTunai(double nominal) {
```

Kata public membuat method dapat dipanggil dari class lain, termasuk Main dan class anak. Tipe kembalian boolean menggantikan void yang sebelumnya dipakai.

Method menerima satu parameter bernama nominal bertipe double. Nilai true dikembalikan jika penarikan sukses, dan false jika penarikan gagal. Komentar di atas method menandai bahwa perubahan ini dibuat pada Praktikum 5.

c.

Pemeriksaan nominal nol atau negative

```
if (nominal <= 0) {
    System.out.println("Gagal: Nominal tarik harus lebih dari 0!");
    return false;
```

Kondisi pertama memeriksa apakah nominal kurang dari atau sama dengan nol.

Jika ya, program menampilkan pesan bahwa nominal penarikan harus lebih dari

0. Pada kondisi ini saldo tidak berubah. Transaksi yang gagal juga tidak dimasukkan ke riwayat. Baris return false memberi tahu pemanggil bahwa penarikan gagal.

d. Pemeriksaan minimum penarikan

```
} else if (nominal < 10000) {
    System.out.println("Transaksi gagal: Minimal nominal penarikan Rp10000.");
    return false;
```

Kondisi else if memeriksa apakah nominal lebih kecil dari 10000. Aturan ini membuat penarikan minimal Rp10.000. Jika nominal terlalu kecil, program menampilkan pesan kegagalan. Saldo tidak dikurangi dan transaksi tidak dicatat.

Method langsung mengembalikan false.

e.

Pemeriksaan saldo

9

```
} else if (nominal > saldo) {
    System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp"+ saldo);
    return false;
```

Kondisi berikutnya memeriksa apakah nominal lebih besar dari saldo. Pada Rekening biasa, saldo tidak boleh menjadi minus. Jika saldo tidak cukup, program menampilkan pesan beserta saldo saat ini. Saldo tetap dan tidak ada transaksi yang dicatat. Method mengembalikan false seperti pada kondisi gagal lainnya.

f.

Penarikan berhasil dan pencatatan transaksi

```
} else {
    saldo -= nominal;

    String idTrx = "TRX-T-" + System.currentTimeMillis();
    Transaksi transaksiBaru = new Transaksi(idTrx,"Debit",nominal);

    riwayatTransaksi.add(transaksiBaru);
```

Blok else dijalankan jika semua pemeriksaan berhasil dilewati. Operator -= mengurangi saldo sesuai nominal penarikan. ID transaksi dibuat dari awalan TRX-T- dan waktu saat ini dengan System.currentTimeMillis(), lalu dipakai untuk membuat object Transaksi baru. Jenis transaksi diisi Debit karena saldo rekening berkurang. Object tersebut kemudian dimasukkan ke riwayatTransaksi dengan perintah add.

g. Mengembalikan nilai true

```
        System.out.println("Tarik tunai Rp" + nominal+ " berhasil. Sisa saldo: Rp" + saldo);

        return true;
    }
}
```

Setelah transaksi dicatat, program menampilkan pesan bahwa tarik tunai berhasil.

Pesan tersebut memuat nominal yang ditarik dan saldo terbaru. Baris return true memberi tahu pemanggil bahwa penarikan sukses. Nilai ini penting karena method transfer harus tahu apakah penarikan berhasil. Tanpa nilai kembalian ini, transfer tidak bisa memutuskan apakah uang boleh disetor ke rekening tujuan.

2.3 Langkah 2 – Overriding tarikTunai pada RekeningGiro (RekeningGiro.java)

Langkah 2 membuat class RekeningGiro menimpa method tarikTunai milik class Rekening. Pada Rekening biasa saldo tidak boleh minus, sedangkan pada Giro saldo boleh minus sampai batas overdraft.

a. Deklarasi class dan atribut

```
1  package pekan5;
2
3  public class RekeningGiro extends Rekening {
4
5      private double batasOverdraft;
```

Kata extends membuat RekeningGiro menjadi class anak dari Rekening. Dengan pewarisan, RekeningGiro otomatis memiliki atribut dan method milik Rekening.

Atribut batasOverdraft hanya dimiliki oleh rekening Giro. Atribut ini menyimpan batas maksimal saldo boleh minus. Keyword private menjaga atribut agar tidak diakses langsung dari luar class.

b. Constructor RekeningGiro

```
public RekeningGiro(String nomor,String nama,double saldoAwal,String pin,double batasOverdraft) {

    super(nomor, nama, saldoAwal, pin);
    this.batasOverdraft = batasOverdraft;
}
```

Constructor menerima nomor, nama, saldo awal, PIN, dan batas overdraft.

Perintah super memanggil constructor milik class Rekening. Data nomor, nama, saldo awal, dan PIN diteruskan ke class induk. Setelah itu nilai batasOverdraft disimpan ke atribut milik RekeningGiro. Dengan cara ini data rekening dasar diurus oleh induk, sedangkan data khusus Giro diurus oleh anak.

c.

Anotasi @Override dan deklarasi method

```
@Override
public boolean tarikTunai(double nominal) {
```

Anotasi @Override memberi tahu compiler bahwa method ini menimpa method milik class induk. Jika nama method salah ketik, compiler akan menampilkan pesan error. Nama method, parameter, dan tipe kembalian sama persis dengan method di class Rekening. Inilah yang disebut overriding atau runtime polymorphism. Saat objeknya berupa RekeningGiro, Java menjalankan versi method milik RekeningGiro.

d. Pemeriksaan nominal dan minimum penarikan

11

```
if (nominal <= 0) {
    System.out.println("Gagal: Nominal tarik Giro harus lebih dari 0!");
    return false;

} else if (nominal < 10000) {
    System.out.println("Transaksi gagal: Minimal nominal penarikan Rp10000.");
    return false;
```

Dua pemeriksaan awal sama dengan yang ada pada class Rekening. Nominal yang kurang dari atau sama dengan nol ditolak. Nominal di bawah Rp10.000 juga ditolak karena melanggar batas minimum penarikan. Kedua kondisi menampilkan pesan gagal dan tidak mengubah saldo. Method mengembalikan false pada kedua kondisi tersebut.

e.

Aturan batas overdraft

```
} else if (nominal > saldo + batasOverdraft) {
    System.out.println("Transaksi Gagal: Melebihi batas overdraft!");

    System.out.println("Maksimal penarikan: Rp" + (saldo + batasOverdraft));

    return false;
```

Kondisi ini adalah aturan khusus Giro yang membedakannya dari Rekening biasa.

Penarikan ditolak hanya jika nominal lebih besar dari saldo ditambah batasOverdraft. Contohnya, saldo Rp100.000 dan overdraft Rp500.000 membuat penarikan maksimal Rp600.000. Jika melebihi batas, program menampilkan pesan dan jumlah penarikan maksimal. Method kemudian mengembalikan false dan saldo tidak berubah.

f.

Penarikan berhasil pada Giro

```
        } else {
            saldo -= nominal;

            String idTrx = "TRX-G-" + System.currentTimeMillis();
            Transaksi transaksiBaru = new Transaksi(idTrx,"Debit Giro",nominal);

            riwayatTransaksi.add(transaksiBaru);

            System.out.println("Tarik tunai Giro Rp" + nominal+ " berhasil. Saldo saat ini: Rp" + saldo);

            return true;
        }
    }
}
```

Blok else dijalankan jika nominal masih berada dalam batas saldo ditambah overdraft. Saldo dikurangi nominal sehingga saldo bisa bernilai negatif. ID transaksi dibuat dari awalan TRX-G- dan waktu saat ini dengan System.currentTimeMillis(). Jenis transaksi diisi Debit Giro agar mudah 12

dibedakan dari Debit pada rekening biasa. Setelah dicatat, program menampilkan pesan berhasil dan mengembalikan true.

g. Getter getBatasOverdraft

```
    public double getBatasOverdraft() {
        return batasOverdraft;
    }
```

Method getBatasOverdraft digunakan untuk membaca nilai batasOverdraft.

Atribut tersebut bersifat private sehingga class lain tidak bisa membacanya langsung. Getter memberi cara yang aman untuk mengambil nilainya. Method ini mengembalikan nilai bertipe double. Kurung kurawal di bawahnya menutup class RekeningGiro.

2.4 Langkah 3 – Uji Coba Pengikatan Dinamis

Langkah 3 menguji pengikatan dinamis dengan menjalankan Main.java. Menurut modul, kode pada Menu 3 (Tarik Tunai) tidak perlu diubah sama sekali. Java akan memilih method tarikTunai yang sesuai dengan jenis objek yang sedang aktif.

a. Variabel bertipe Rekening

```
ArrayList<Rekening> daftarRekening =new ArrayList<Rekening>();
Rekening akunAktif = null;
```

Variabel akunAktif bertipe Rekening, yaitu class induk. Variabel ini dapat berisi objek RekeningTabungan maupun RekeningGiro. ArrayList daftarRekening juga bertipe Rekening sehingga dapat menyimpan kedua jenis rekening. Awalnya akunAktif bernilai null karena belum ada rekening yang dipilih. Ini adalah bentuk polimorfisme, yaitu satu tipe variabel yang bisa menampung banyak bentuk objek.

b. Membuat objek sesuai pilihan produk

```
Rekening rekeningBaru;

if (produk == 1) {

    System.out.print("Masukkan Suku Bunga (%): ");
    double sukuBunga = input.nextDouble();
    input.nextLine();

    rekeningBaru = new RekeningTabungan(no, nama, saldo, pin, sukuBunga);

} else if (produk == 2) {

    System.out.print("Masukkan Batas Overdraft: ");
    double batasOverdraft = input.nextDouble();
    input.nextLine();

    rekeningBaru = new RekeningGiro(no, nama, saldo, pin, batasOverdraft);

} else {

    System.out.println("Pilihan produk tidak valid!");
    break;
}

daftarRekening.add(rekeningBaru);
akunAktif = rekeningBaru;
```

Variabel rekeningBaru bertipe Rekening, tetapi isinya ditentukan oleh pilihan produk. Jika pengguna memilih produk 1, program membuat object RekeningTabungan. Jika pengguna memilih produk 2, program membuat object RekeningGiro dan meminta batas overdraft. Kedua object dapat disimpan ke variabel bertipe Rekening karena sama-sama class anak dari Rekening. Object tersebut lalu dimasukkan ke daftarRekening dan dijadikan akunAktif.

c.

Memanggil tarikTunai pada Menu 3

```
System.out.print("Masukkan PIN: ");
String pinTarik = input.nextLine();

if (akunAktif.otentikasi(pinTarik)) {

    System.out.print("Masukkan nominal tarik: ");
    double tarik = input.nextDouble();
    input.nextLine();

    akunAktif.tarikTunai(tarik);
```

Program meminta PIN lalu memeriksanya dengan method otentikasi. Jika PIN benar,pengguna memasukkan suatu nominal penarikan.Pada Baris akunAktif.tarikTunai(tarik) tidak mengecek jenis rekening dengan if-else. Saat program berjalan, Java melihat isi akunAktif lalu memilih method tarikTunai yang sesuai. Jika isinya RekeningGiro, versi Giro yang dijalankan, sedangkan Tabungan memakai versi milik Rekening.

d. Skenario uji coba dari modul

Uji coba mengikuti petunjuk modul, yaitu membuat akun baru dengan produk Giro. Saldo awal diisi Rp100.000 dan batas overdraft diisi Rp500.000. Kemudian dilakukan tarik tunai sebesar Rp400.000 melalui Menu 3. Pada Rekening biasa penarikan ini pasti ditolak karena melebihi saldo. Pada RekeningGiro penarikan berhasil dan saldo menjadi minus Rp300.000, seperti terlihat pada Pengujian 3.

B. TUGAS

2.5 Tugas 1 – Method transfer pada Class Rekening

Tugas 1 meminta penambahan method transfer pada class Rekening. Method ini memakai parameter bertipe Rekening sehingga rekening tujuan boleh berupa Tabungan atau Giro. Urutan logikanya adalah otentikasi PIN, tarik tunai dari rekening asal, lalu setor tunai ke rekening tujuan jika penarikan berhasil.

a. Deklarasi method transfer

```
public void transfer(Rekening tujuan, double nominal, String pinInput) {
```

Method transfer menerima tiga parameter, yaitu tujuan, nominal, dan pinInput.

Parameter tujuan bertipe Rekening, yaitu class induk. Karena itu parameter ini dapat menerima object RekeningTabungan maupun RekeningGiro. Inilah yang disebut parameter polimorfik. Method ini bertipe void karena hasilnya langsung ditampilkan lewat pesan.

b. Otentikasi PIN pengirim

```
if (!otentikasi(pinInput)) {
    System.out.println("Transfer gagal: PIN yang Anda masukkan salah!");
    return;
}
```

Langkah pertama pada transfer adalah memeriksa PIN pengirim. Method otentikasi membandingkan pinInput dengan PIN milik rekening asal. Tanda seru membalik hasilnya, sehingga blok if berjalan jika PIN salah. Jika PIN salah, program menampilkan pesan transfer gagal. Perintah return langsung menghentikan method sehingga tidak ada uang yang dipindahkan.

c.

Pemeriksaan rekening tujuan

15

```
if (tujuan == null) {
    System.out.println("Transfer gagal: Rekening tujuan tidak ditemukan.");
    return;
}
```

Bagian ini memeriksa apakah rekening tujuan bernilai null. Nilai null berarti rekening tujuan tidak ditemukan. Jika null, program menampilkan pesan transfer gagal. Pemeriksaan ini mencegah error saat program memanggil method milik rekening tujuan. Method dihentikan dengan return tanpa mengubah saldo.

d. Pemeriksaan transfer ke rekening sendiri

```
if (this == tujuan) {
    System.out.println("Transfer gagal: Tidak dapat transfer ke rekening sendiri.");
    return;
}
```

Kondisi this == tujuan membandingkan object pengirim dengan object tujuan.

Kata this menunjuk pada rekening yang sedang melakukan transfer. Jika keduanya sama, berarti pengguna mencoba transfer ke rekening sendiri. Program menampilkan pesan bahwa hal tersebut tidak dapat dilakukan. Method dihentikan agar saldo dan riwayat tidak berubah tanpa alasan.

e.

Tarik tunai lalu setor tunai

```
    if (this.tarikTunai(nominal)) {
       tujuan.setorTunai(nominal);

       System.out.println("Transfer berhasil ke rekening "+ tujuan.getNomorRekening() + ".");
    } else {
       System.out.println("Transfer gagal.");
    }
}
```

Perintah this.tarikTunai(nominal) menarik uang dari rekening pengirim terlebih dahulu. Karena tarikTunai di-override, Giro memakai aturan overdraft dan Tabungan memakai aturan biasa. Hasil boolean dari tarikTunai diperiksa oleh if.

Jika hasilnya true, tujuan.setorTunai(nominal) menyetor uang ke rekening tujuan lalu pesan transfer berhasil ditampilkan. Jika hasilnya false, program hanya menampilkan pesan transfer gagal sehingga saldo rekening tujuan tidak berubah.

2.6 Tugas 2 – Integrasi Menu 8 pada Class Main

Tugas 2 menambahkan menu 8 pada class Main. Menu ini meminta nomor rekening tujuan, mencari rekening tersebut di daftarRekening, lalu memanggil method transfer.

a. Menambahkan menu 8

```
System.out.println("7. Simulasi Akhir Bulan (Khusus Tabungan)");
System.out.println("8. Transfer ke Rekening Lain");
System.out.println("9. Keluar");
```

Baris ini menampilkan pilihan Transfer ke Rekening Lain pada menu utama.

Nomor 8 dipakai untuk fitur transfer yang baru dibuat. Pilihan Keluar berada pada nomor 9. Nomor 7 tetap dipakai oleh Simulasi Akhir Bulan. Semua pilihan tersebut dijalankan melalui switch-case pada method main.

b. Case 8 dan input rekening tujuan

```
case 8:
    if (akunAktif == null) {

        System.out.println("Error: Anda belum membuka rekening!");

        break;
    }

    System.out.print("Masukkan Nomor Rekening Tujuan: ");

    String nomorTujuan = input.nextLine();
```

Case 8 dijalankan ketika pengguna memilih menu Transfer ke Rekening Lain.

Program memeriksa apakah akunAktif masih null. Jika belum ada rekening, program menampilkan pesan error. Perintah break menghentikan case sehingga proses transfer tidak dilanjutkan. Jika ada rekening aktif, program meminta nomor rekening tujuan dan menyimpannya di nomorTujuan.

c.

Mencari rekening tujuan di daftarRekening

```
Rekening akunTujuan = null;

for (Rekening rekening : daftarRekening) {

    if (rekening.getNomorRekening().equals(nomorTujuan)) {

        akunTujuan = rekening;
        break;
    }
}
```

Variabel akunTujuan dibuat dengan nilai awal null. Perulangan for-each mengambil object Rekening satu per satu dari daftarRekening. Setiap nomor rekening dibandingkan dengan nomorTujuan menggunakan method equals. Jika sama, object tersebut disimpan ke akunTujuan. Perintah break menghentikan pencarian karena rekening yang dicari sudah ditemukan.

d. Memanggil method transfer

```
if (akunTujuan != null) {

    System.out.print("Masukkan nominal transfer: ");

    double nominal = input.nextDouble();
    input.nextLine();

    System.out.print("Masukkan PIN: ");
    String pinTransfer = input.nextLine();

    akunAktif.transfer(akunTujuan,nominal,pinTransfer);
```

Jika akunTujuan tidak null, program meminta nominal transfer. Program juga meminta PIN milik rekening pengirim. Method transfer dipanggil pada akunAktif dengan akunTujuan, nominal, dan PIN. Pemanggilan ini tidak peduli apakah akunTujuan berupa Tabungan atau Giro. Seluruh pemeriksaan dan pemindahan uang dilakukan di dalam method transfer.

e.

Rekening tujuan tidak ditemukan

```
} else {
    System.out.println("Rekening tujuan tidak ditemukan.");
}
break;
```

Bagian else dijalankan jika rekening tujuan tidak ditemukan di daftarRekening.

Program menampilkan pesan bahwa rekening tujuan tidak ditemukan. Pada 18

kondisi ini nominal dan PIN tidak diminta. Method transfer tidak dipanggil sehingga tidak ada saldo yang berubah. Perintah break menutup case 8 dan program kembali ke menu utama.

2.7 Kelengkapan Program pada Rekening Tabungan dan Main

Bagian ini menjelaskan class dan menu lain yang ikut dijalankan pada program Praktikum 5. Class RekeningTabungan menjadi pasangan RekeningGiro sebagai class anak dari Rekening.

a. Class RekeningTabungan

```
1  package pekan5;
2
3  public class RekeningTabungan extends Rekening {
4
5      private double sukuBunga;
6
7      public RekeningTabungan( String nomor,String nama,double saldoAwal,String pin,double sukuBunga) {
8
9          super(nomor, nama, saldoAwal, pin);
10         this.sukuBunga = sukuBunga;
11     }
12
13     public double getSukuBunga() {
14         return sukuBunga;
15     }
16
17     public void tambahBungaAkhirBulan() {
18         double bunga = saldo * sukuBunga / 100;
19         saldo += bunga;
20
21         System.out.println("Bunga akhir bulan sebesar Rp" + bunga+ " telah ditambahkan.");
22
23         System.out.println("Saldo setelah bunga: Rp" + saldo);
24     }
25 }
```

RekeningTabungan juga memakai extends Rekening sehingga menjadi class anak seperti RekeningGiro. Atribut sukuBunga menyimpan persentase bunga khusus rekening tabungan. Constructor memanggil super lalu menyimpan nilai sukuBunga. Class ini tidak meng-override tarikTunai sehingga memakai versi milik Rekening. Method tambahBungaAkhirBulan menghitung bunga dari saldo dikali sukuBunga dibagi 100, lalu menambahkannya ke saldo.

b. Menu 7 dengan instanceof dan casting

```
} else if (akunAktif instanceof RekeningTabungan) {
    RekeningTabungan tabungan =(RekeningTabungan) akunAktif;
    tabungan.tambahBungaAkhirBulan();
} else {
    System.out.println("Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan.");
}
break;
```

Menu 7 hanya berlaku untuk rekening tabungan. Operator instanceof memeriksa apakah akunAktif berisi object RekeningTabungan. Jika benar, akunAktif diubah tipenya (casting) menjadi RekeningTabungan. Casting diperlukan karena method tambahBungaAkhirBulan tidak dimiliki class Rekening. Jika akunAktif berupa Giro, program menampilkan pesan bahwa fitur ini hanya untuk Rekening Tabungan.

c.

Menu 9 dan keluar dari program

```
case 9:
    isRunning = false;
    System.out.println("Sistem ditutup. Terima kasih!" );
    break;
```

Menu 9 dipakai untuk mengakhiri program. Saat dipilih, nilai isRunning diubah menjadi false.Perulangan while berhenti karena syaratnya tidak terpenuhi lagi.

Program menampilkan pesan bahwa sistem ditutup. Setelah perulangan selesai, object Scanner ditutup dengan input.close().

C. HASIL PENGUJIAN

2.8 Hasil Pengujian Praktikum

Pengujian dilakukan untuk memastikan menu dan fitur pada kelima class dapat bekerja sesuai dengan kode. Hasil yang ditampilkan berasal dari program yang dijalankan dengan data rekening Tabungan 1234567 milik lexi dan rekening Giro 789000 milik caca.

a. Pengujian 1 – Membuka Rekening Tabungan

```
==SISTEM PERBANKAN MINI==

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
9. Keluar
Pilih menu: 1
Masukkan No Rekening: 1234567
Masukkan Nama Pemilik: lexi
Masukkan Saldo Awal: 5000000
Masukkan PIN (6 digit): 123456

Pilih Produk:
1. Tabungan Umum
2. Giro Bisnis
Pilihan produk: 1
Masukkan Suku Bunga (%): 2
Rekening atas nama lexi berhasil dibuat dengan saldo Rp5000000.0
```

Menu 1 dipilih untuk membuat rekening baru. Data yang dimasukkan adalah nomor 1234567, nama lexi, saldo awal 5000000, dan PIN 123456. Pada pilihan produk dipilih nomor 1, yaitu Tabungan Umum dengan suku bunga 2. Program menampilkan bahwa rekening atas nama lexi berhasil dibuat dengan saldo Rp5.000.000. Hasil ini menunjukkan object RekeningTabungan berhasil dibuat.

b. Pengujian 2 – Membuka Rekening Giro

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
9. Keluar
Pilih menu: 1
Masukkan No Rekening: 89012345
Masukkan Nama Pemilik: caca
Masukkan Saldo Awal: 1000000
Masukkan PIN (6 digit): 123456

Pilih Produk:
1. Tabungan Umum
2. Giro Bisnis
Pilihan produk: 2
Masukkan Batas Overdraft: 500000
Rekening atas nama caca berhasil dibuat dengan saldo Rp1000000.0
```

Menu 1 dipilih lagi untuk membuat rekening kedua. Datanya adalah nomor 789000, nama caca, saldo awal 100000, dan PIN 123456. Pada pilihan produk dipilih nomor 2, yaitu Giro Bisnis dengan batas overdraft 500000. Program menampilkan bahwa rekening atas nama caca berhasil dibuat dengan saldo 21

Rp100.000. Rekening Giro ini otomatis menjadi akun aktif, dan hasilnya menunjukkan object RekeningGiro berhasil dibuat.

c.

Pengujian 3 – Tarik Tunai Giro Melebihi Saldo (Dynamic Dispatch)

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
9. Keluar
Pilih menu: 3
Masukkan PIN: 123456
Masukkan nominal tarik: 400000
Tarik tunai Giro Rp400000.0 berhasil. Saldo saat ini: Rp600000.0
```

Akun aktif adalah rekening Giro milik caca dengan saldo Rp100.000. Menu 3 dipilih, PIN 123456 dimasukkan, lalu nominal penarikan Rp400.000 dimasukkan.

Nominal ini lebih besar dari saldo, tetapi masih di bawah saldo ditambah overdraft, yaitu Rp600.000. Program menampilkan pesan tarik tunai Giro berhasil dan saldo menjadi minus Rp300.000. Hasil ini membuktikan dynamic method dispatch karena Java menjalankan method tarikTunai milik RekeningGiro.

d. Pengujian 4 – Tarik Tunai Giro Melebihi Batas Overdraft

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
9. Keluar
Pilih menu: 3
Masukkan PIN: 123456
Masukkan nominal tarik: 3000000
Transaksi Gagal: Melebihi batas overdraft!
Maksimal penarikan: Rp1100000.0
```

Menu 3 dipilih dengan PIN yang benar pada rekening Giro yang saldonya minus Rp300.000. Nominal penarikan yang dimasukkan adalah Rp300.000. Saldo ditambah batas overdraft sekarang hanya Rp200.000. Program menolak penarikan dan menampilkan pesan melebihi batas overdraft beserta maksimal penarikan 22

Rp200.000. Hasil ini menunjukkan aturan overdraft pada RekeningGiro berjalan dengan benar.

e.

Pengujian 5 – Tarik Tunai Tabungan Melebihi Saldo

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
9. Keluar
Pilih menu: 5
Masukkan nomor rekening: 1234567
Akun berhasil diganti.

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
9. Keluar
Pilih menu: 3
Masukkan PIN: 123456
Masukkan nominal tarik: 7000000
Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp5000000.0
```

Menu 5 dipilih untuk berpindah ke rekening Tabungan nomor 1234567, lalu program menampilkan pesan akun berhasil diganti. Setelah itu Menu 3 dipilih dengan PIN yang benar dan nominal Rp6.000.000. Saldo Tabungan hanya Rp5.000.000 sehingga penarikan ditolak. Program menampilkan pesan saldo tidak mencukupi. Hasil ini menunjukkan bahwa menu yang sama menghasilkan perilaku berbeda pada Tabungan dan Giro, yaitu polimorfisme.

f.

Pengujian 6 – Transfer Berhasil

23

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
9. Keluar
Pilih menu: 8
Masukkan Nomor Rekening Tujuan: 89012345
Masukkan nominal transfer: 1000000
Masukkan PIN: 123456
Tarik tunai Rp1000000.0 berhasil. Sisa saldo: Rp4000000.0
Setor tunai Rp1000000.0 berhasil. Saldo saat ini: Rp1600000.0
Transfer berhasil ke rekening 89012345.
```

Akun aktif adalah rekening Tabungan 1234567 milik lexi dengan saldo Rp5.000.000. Menu 8 dipilih dengan tujuan rekening 789000, nominal Rp1.000.000, dan PIN 123456. Program menarik Rp1.000.000 dari Tabungan sehingga saldonya menjadi Rp4.000.000. Uang tersebut disetor ke rekening Giro sehingga saldonya naik dari minus Rp300.000 menjadi Rp700.000. Program menampilkan pesan transfer berhasil ke rekening 789000, sehingga transfer ke rekening Giro dapat dilakukan.

g. Pengujian 7 – Transfer dengan PIN Salah

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
9. Keluar
Pilih menu: 8
Masukkan Nomor Rekening Tujuan: 89012345
Masukkan nominal transfer: 500000
Masukkan PIN: 3458973
Transfer gagal: PIN yang Anda masukkan salah!
```

Pada pengujian ini, pengguna memilih menu 8 dan memasukkan nomor rekening tujuan 89012345.Nominal transfer yang dimasukkan adalah Rp500.000.PIN yang dimasukkan salah sehingga program menolak transaksi. Program menampilkan pesan bahwa PIN tidak sesuai. Karena PIN salah, transfer tidak dilakukan dan saldo rekening tidak berubah.

h. Pengujian 8 – Rekening Tujuan Tidak Ditemukan

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
9. Keluar
Pilih menu: 8
Masukkan Nomor Rekening Tujuan: 123890
Rekening tujuan tidak ditemukan.
```

Pada pengujian ini, pengguna memilih menu 8 dan memasukkan nomor rekening tujuan 123890.Nomor tersebut tidak ditemukan dalam daftar rekening yang tersedia.Program langsung menampilkan pesan bahwa rekening tujuan tidak ditemukan.Nominal transfer dan PIN tidak diminta karena rekening tujuan tidak valid. Transaksi dihentikan sehingga tidak ada saldo yang berubah.

i.

Pengujian 9 – Transfer ke Rekening Sendiri

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
9. Keluar
Pilih menu: 8
Masukkan Nomor Rekening Tujuan: 1234567
Masukkan nominal transfer: 200000
Masukkan PIN: 123456
Transfer gagal: Tidak dapat transfer ke rekening sendiri.
```

Pada pengujian ini,nomor rekening tujuan yang dimasukkan adalah 1234567.Nomor tersebut sama dengan nomor rekening yang sedang digunakan sebagai pengirim.

Program mendeteksi bahwa transfer dilakukan ke rekening sendiri. Transaksi ditolak dan program menampilkan pesan kegagalan. Saldo rekening tetap karena uang tidak dipindahkan.

j. Pengujian 10 – Transfer dengan Saldo Tidak Cukup

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
9. Keluar
Pilih menu: 8
Masukkan Nomor Rekening Tujuan: 89012345
Masukkan nominal transfer: 9000000
Masukkan PIN: 123456
Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp4000000.0
Transfer gagal.
```

Pada pengujian ini, pengguna memasukkan nominal transfer sebesar Rp9.000.000. Saldo rekening pengirim hanya Rp4.000.000 sehingga saldo tidak mencukupi.Program menampilkan pesan bahwa saldo tidak mencukupi dan transfer gagal. Penarikan uang dari rekening pengirim tidak berhasil dilakukan.

Karena itu, rekening tujuan tidak menerima uang dan saldo tidak berubah.

k. Pengujian 11 – Transfer di Bawah Rp10.000

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
9. Keluar
Pilih menu: 8
Masukkan Nomor Rekening Tujuan: 89012345
Masukkan nominal transfer: 5000
Masukkan PIN: 123456
Transaksi gagal: Minimal nominal penarikan Rp10000.
Transfer gagal.
```

Pada pengujian ini, nominal transfer yang dimasukkan adalah Rp5.000. PIN yang dimasukkan benar, tetapi nominal tersebut lebih kecil dari batas minimum Rp10.000. Program menolak transaksi sesuai aturan minimal penarikan yang berlaku. Pesan kegagalan ditampilkan agar pengguna mengetahui penyebab transaksi ditolak. Tidak ada uang yang dipindahkan pada pengujian ini.

l. Pengujian 12 – Transfer dari Giro Memakai Overdraft

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
9. Keluar
Pilih menu: 5
Masukkan nomor rekening: 89012345
Akun berhasil diganti.

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
9. Keluar
Pilih menu: 8
Masukkan Nomor Rekening Tujuan: 1234567
Masukkan nominal transfer: 1000000
Masukkan PIN: 123456
Tarik tunai Giro Rp1000000.0 berhasil. Saldo saat ini: Rp600000.0
Setor tunai Rp1000000.0 berhasil. Saldo saat ini: Rp5000000.0
Transfer berhasil ke rekening 1234567.
```

Pada pengujian ini, pengguna berpindah ke rekening Giro lalu melakukan transfer sebesar Rp1.000.000 ke rekening 1234567. Penarikan berhasil karena nominal masih berada dalam batas overdraft rekening Giro. Setelah uang ditarik, program menyetorkan nominal tersebut ke rekening tujuan. Program kemudian menampilkan pesan bahwa transfer berhasil. Hasil ini menunjukkan bahwa fitur transfer dapat digunakan dari rekening Giro.

m. Pengujian 13 – Cetak Mutasi Rekening Giro

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
9. Keluar
Pilih menu: 6
Masukkan PIN: 123456
--- MUTASI REKENING ---
ID Transaksi : TRX-G-1792935736856 | Jenis : Debit Giro | Nominal : Rp400000.0
ID Transaksi : TRX-S-2 | Jenis : Kredit | Nominal : Rp1000000.0
ID Transaksi : TRX-G-1792936186427 | Jenis : Debit Giro | Nominal : Rp1000000.0
```

Pada pengujian ini, pengguna memilih menu 6 untuk mencetak mutasi rekening Giro. Program meminta PIN dan pengguna memasukkan PIN yang benar.

Program menampilkan daftar transaksi yang tercatat pada rekening tersebut.

Setiap transaksi menunjukkan ID, jenis transaksi, dan nominalnya. Hasil ini menunjukkan bahwa riwayat transaksi Giro dapat dilihat melalui menu mutasi.

n. Pengujian 14 – Cetak Mutasi Rekening Tabungan

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
9. Keluar
Pilih menu: 5
Masukkan nomor rekening: 1234567
Akun berhasil diganti.

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
9. Keluar
Pilih menu: 6
Masukkan PIN: 123456
--- MUTASI REKENING ---
ID Transaksi : TRX-T-1792935927036 | Jenis : Debit | Nominal : Rp1000000.0
ID Transaksi : TRX-S-2 | Jenis : Kredit | Nominal : Rp1000000.0
```

Pada pengujian ini, pengguna terlebih dahulu mengganti akun ke rekening Tabungan nomor 1234567. Setelah itu, menu 6 dipilih dan PIN yang benar dimasukkan. Program menampilkan riwayat transaksi pada rekening Tabungan.

Riwayat tersebut mencakup transaksi debit dan kredit beserta nominalnya.

Dengan demikian, pengguna dapat memeriksa uang yang keluar dan masuk dari rekening Tabungan.

o. Pengujian 15 – Cek Informasi Rekening

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
9. Keluar
Pilih menu: 4
--- INFO REKENING ---
No. Rekening : 1234567
Nama Pemilik : lexi
Saldo Akhir  : Rp5000000.0
```

Pada pengujian ini, pengguna memilih menu 4 untuk melihat informasi rekening aktif. Program menampilkan nomor rekening 1234567 dan nama pemilik 28

rekening, yaitu lexi. Saldo akhir yang ditampilkan adalah Rp5.000.000. Informasi tersebut diambil dari data rekening yang sedang aktif. Hasil ini menunjukkan bahwa menu cek informasi rekening berjalan dengan baik.

2.9 Rangkuman Latihan dan Tugas

Latihan membahas pembuatan package pekan5, perubahan method tarikTunai pada class Rekening menjadi bertipe boolean, overriding method tarikTunai pada class RekeningGiro, serta uji coba pengikatan dinamis pada Menu 3. Atribut saldo dan riwayatTransaksi dibuat protected agar dapat dipakai oleh class anak. Dengan overriding, rekening Giro dapat ditarik melebihi saldo sampai batas overdraft, sedangkan rekening Tabungan tetap memakai aturan Rekening biasa.

Tugas membahas pembuatan method transfer pada class Rekening dan penambahan Menu 8 pada class Main. Method transfer memakai parameter bertipe Rekening sehingga rekening tujuan dapat berupa Tabungan maupun Giro. Transfer hanya berhasil jika PIN benar, rekening tujuan ada, bukan rekening sendiri, dan penarikan dari rekening asal berhasil. Transaksi yang gagal tidak masuk ke riwayat sehingga mutasi hanya berisi transaksi yang berhasil.

BABIII
PENUTUP

3.1 Kesimpulan

Praktikum 5 menunjukkan penerapan polimorfisme melalui class Rekening, RekeningTabungan, RekeningGiro, Transaksi, dan Main. Method tarikTunai yang ditimpa pada RekeningGiro membuat satu pemanggilan method yang sama menghasilkan perilaku berbeda sesuai tipe objek aslinya. Java memilih method yang tepat saat program berjalan melalui dynamic method dispatch, tanpa perlu if-else untuk mengecek jenis rekening.

Method transfer dengan parameter bertipe Rekening dapat menerima rekening Tabungan maupun Giro. Hasil pengujian menunjukkan transfer berhasil jika semua syarat terpenuhi, dan gagal dengan pesan yang sesuai jika PIN salah, rekening tujuan tidak ditemukan, rekening sama, nominal terlalu kecil, atau saldo tidak cukup.

3.2 Saran

Program dapat dikembangkan dengan menambahkan penyimpanan data secara permanen, tanggal dan waktu transaksi, pembatasan jumlah percobaan PIN, serta pencatatan yang membedakan transfer masuk dan transfer keluar. Selain itu, program dapat diberi penanganan input yang salah, misalnya ketika pengguna memasukkan huruf pada saat diminta angka, agar program tidak berhenti karena error.

DAFTAR PUSTAKA

Nurfiah. Modul Praktikum Pemrograman Berorientasi Objek – Java. Praktikum 5: Transaksi Dinamis (Polymorphism).

Oracle. (2023). Java Documentation. Dokumentasi mengenai class, object, inheritance, overriding, polymorphism, ArrayList, Scanner, constructor, method, dan struktur kontrol Java.

Eckel, B. (2006). Thinking in Java. Prentice Hall.

Sierra, K., & Bates, B. (2005). Head First Java. O'Reilly Media.